Arrays
2D arrays
Arraylist, 2D
Queues
Recursion
Trees → BFS, DFS
Maps
OOPS
Heaps

Practice → A Lot
Code khud se likho

PW SKILLS

Graphs

Checklist

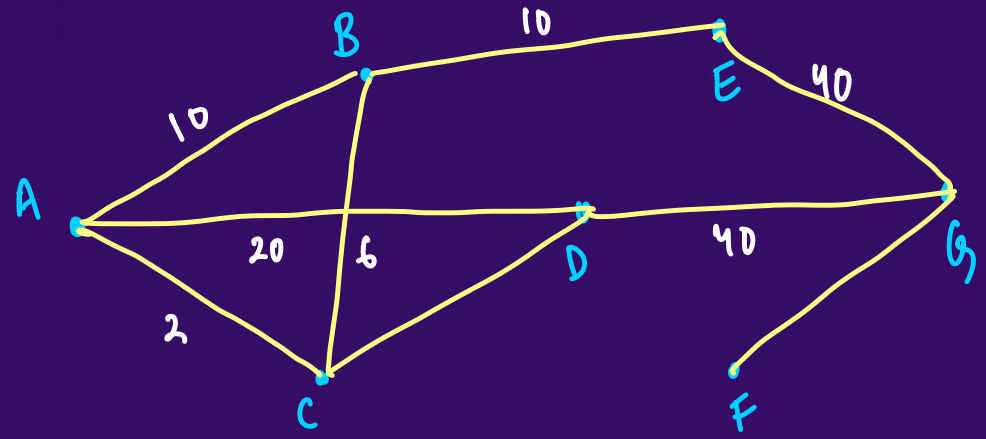
1. Visualization & Terminology
2. Representation / Storage
3. BFS and DFS
4. Topological Sort
5. Dijkstra's Algorithm
6. ~~MCM problems and DP on Trees~~

& so on

Edges, Vertices, Weight, Direction

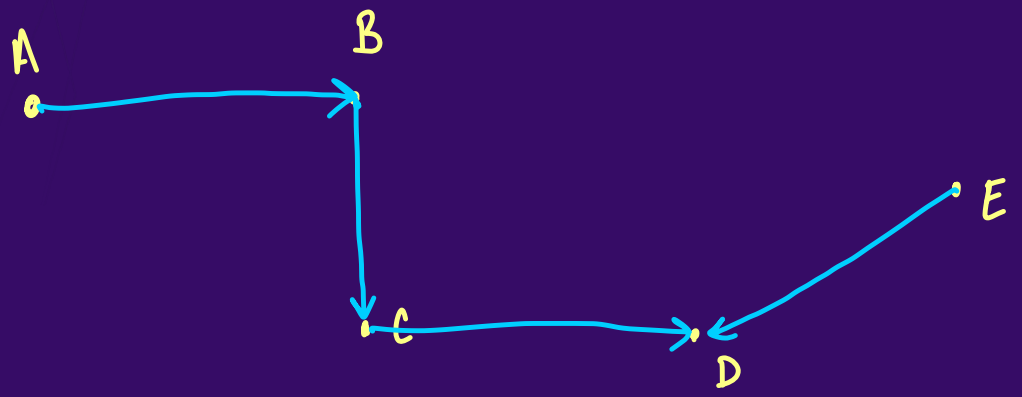
Google Maps

Shortest Dist
Shortest Time

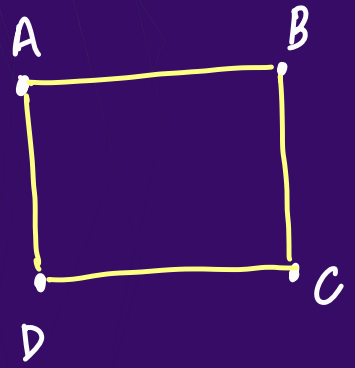


Edges, Vertices, Weight, Direction

Directed Graphs (one way Roads)

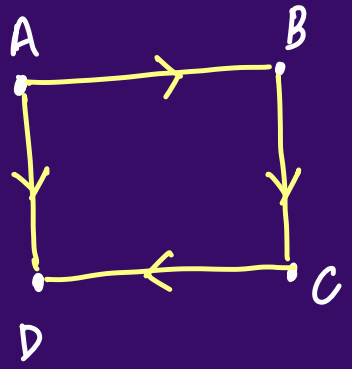


Cycle & Connected Components



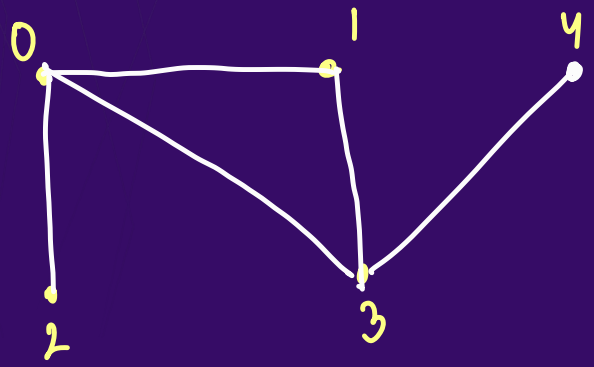
Cycle → A → B → C → D → A

No cycle

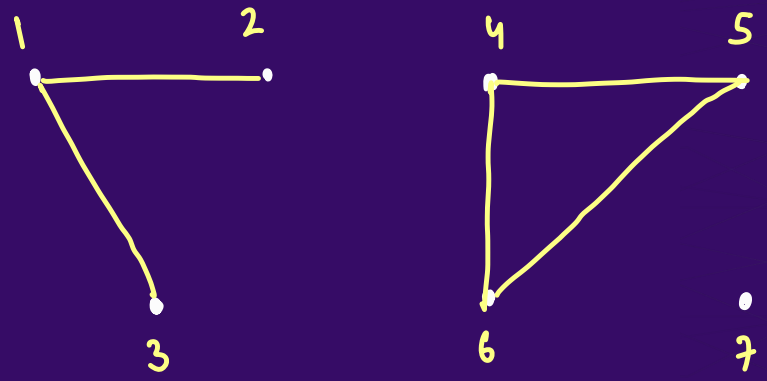


No cycle

Cycle & Connected Components



1 component



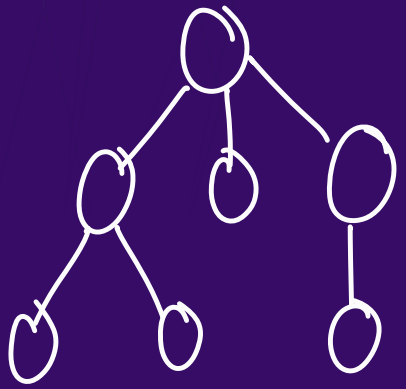
3 components

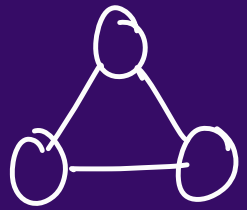
1, 2, 3 4, 5, 6 7

Representation of Graph

Any tree is a graph.

But any graph may or may not be a tree



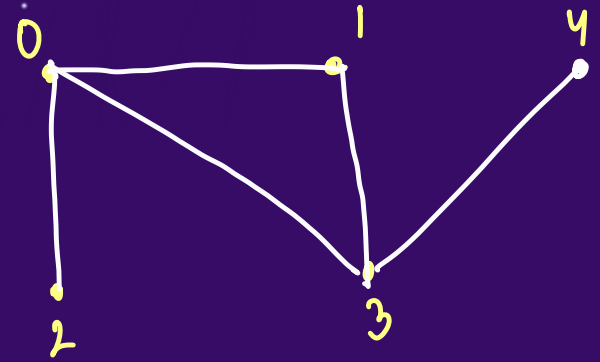


Representation of Graph

We have 2 ways : 1) Adjacency Matrix → 0's, 1's

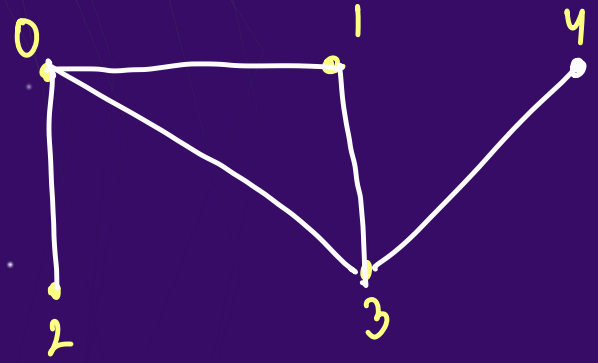
2) Adjacency List → to save space,

For Ex :



	0	1	2	3	4
0	∝	1	1	1	0
1	1	∝	0	1	0
2	1	0	∝	0	0
3	1	1	0	∝	1
4	0	0	0	1	∝

Representation of Graph



List < List <Integer>> list ;

0 : {1, 2, 3}

1 : {0, 3}

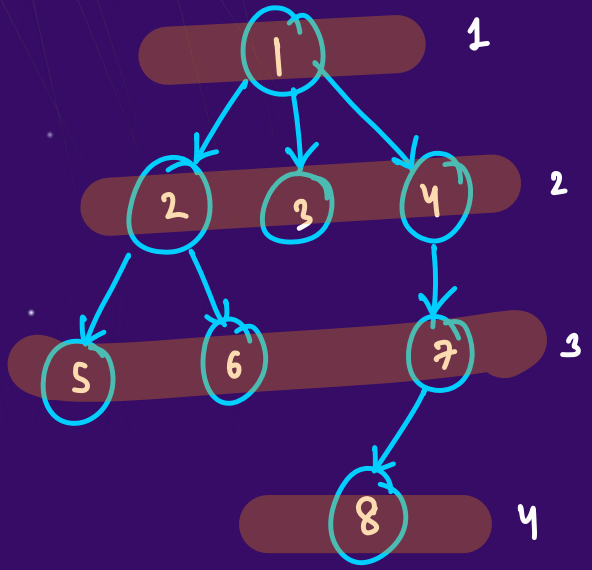
2 : {0}

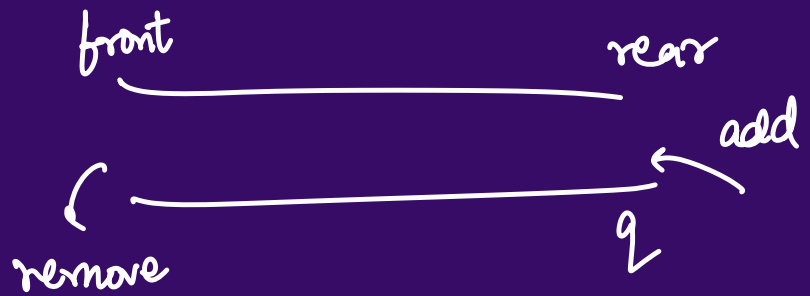
3 : {0, 1, 4}

4 : {3}

adj = { {0, 1, 3}, {0, 3}, {0}, {0, 1, 4}, {3} }

Breadth First Search (Queue)

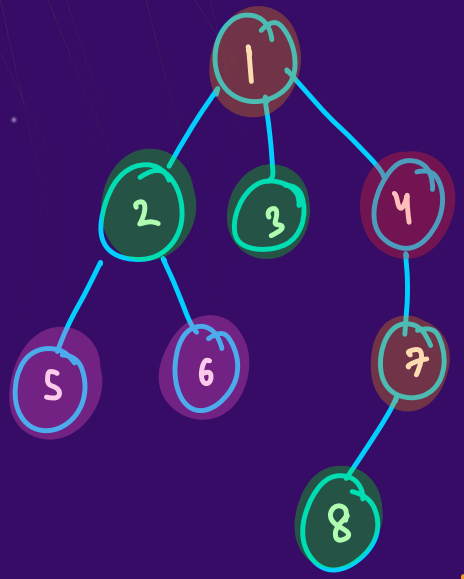




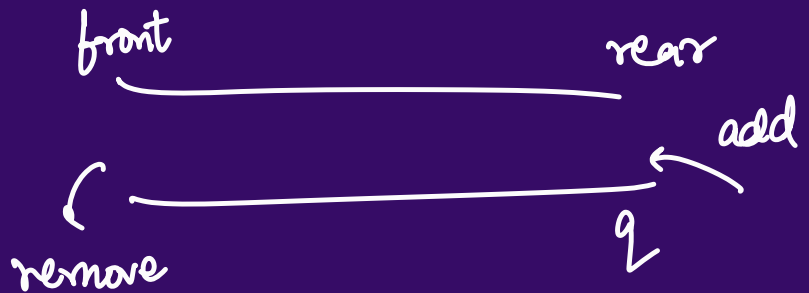
Tree

1 2 3 4 5 6 7 8

Breadth First Search



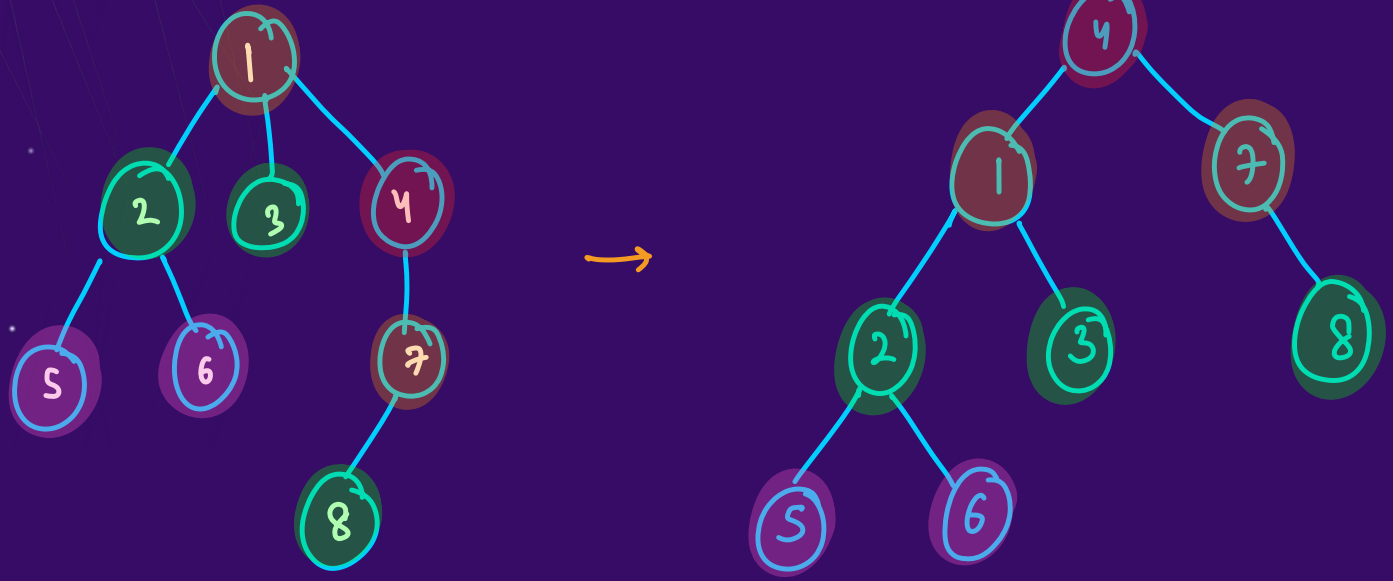
Graph



BFS: 4 1 7 2 3 8 5 6

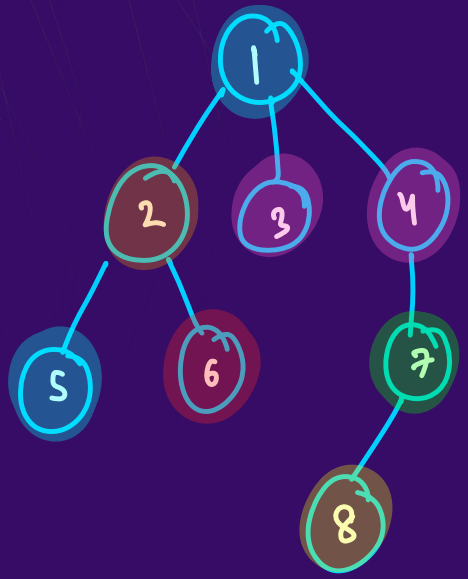
(about 4)

Breadth First Search



4 1 7 2 3 8 5 6

Breadth First Search



front — rear

remove ← — → add

1 →	4	1	7	2	3	8	5	6
6 →	6	2	1	5	3	4	7	8

visit =

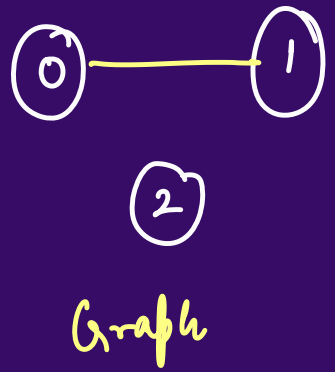
0	1	2	3	4	5	6	7	8
F	T	T	T	T	T	T	T	T

Ques:

Q : Number of Provinces

Hint : BFS

	0	1	2
0	1	1	0
1	1	1	0
2	0	0	1



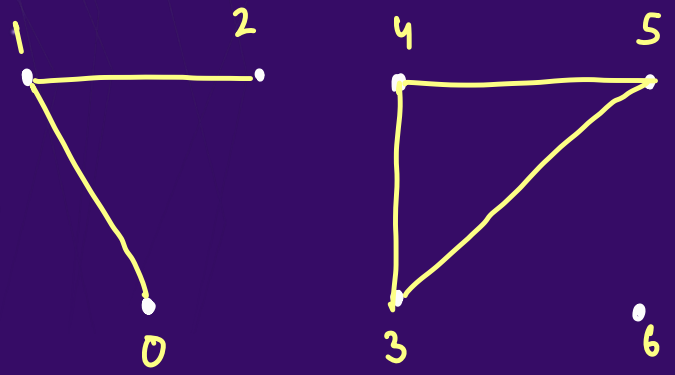
Graph

No. of provinces = 2

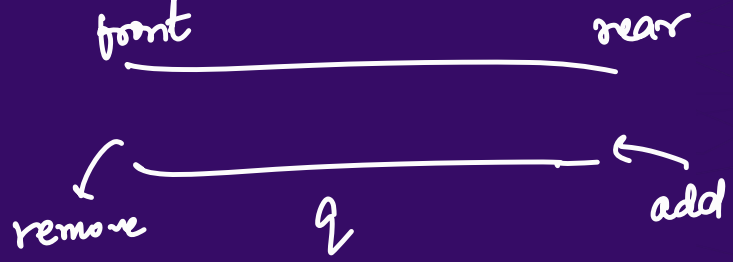
[Leetcode 547]

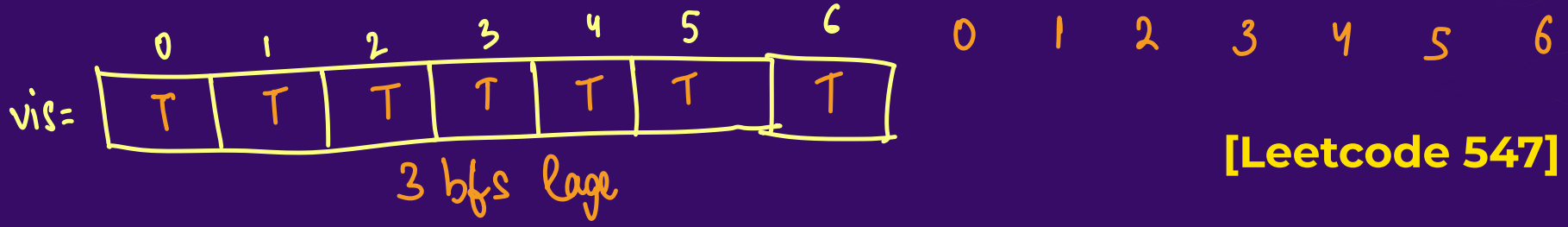
Ques: count = 0 1 2 3

Q : Number of Provinces (BFS)




```
for(i = 0 to 6)
    if(!vis[i]){
        bfs(i)
        count ++
    }
```

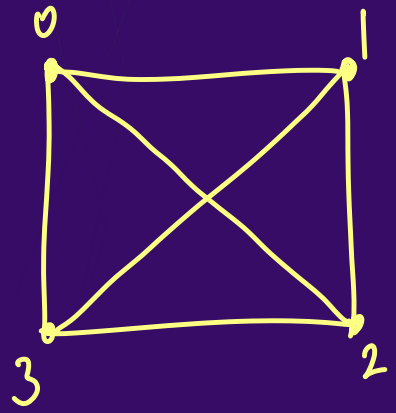




[Leetcode 547]

Ques:

Q : Number of Provinces (BFS)



0 1 2 3

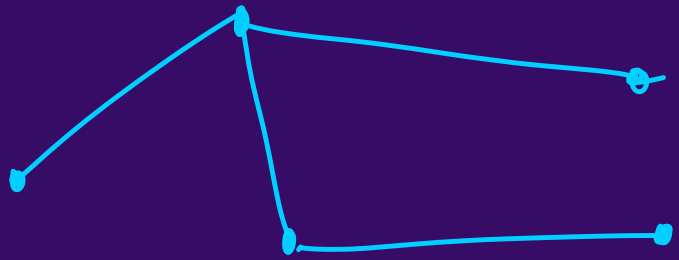
Worst Case : $O(n^2)$ T.C. → adj matrix

Every Case : →

Adj. list

↓

$O(V + 2E)$



[Leetcode 547]

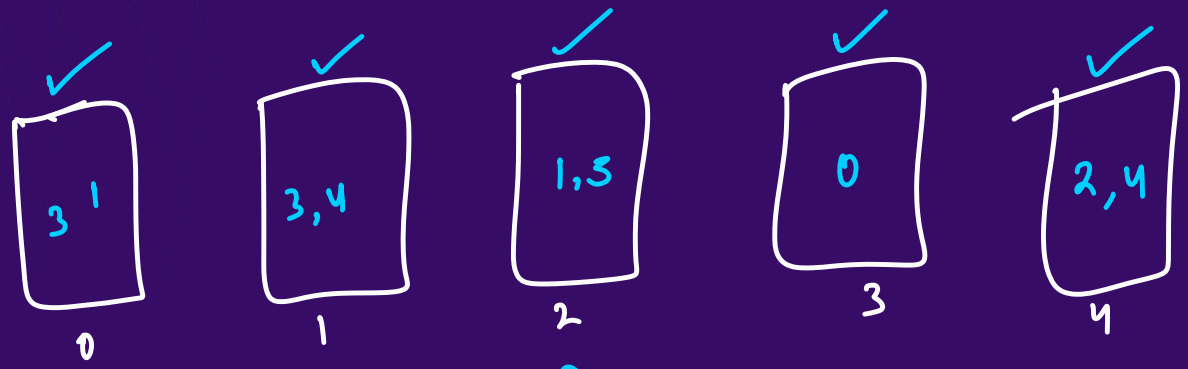
Ques:

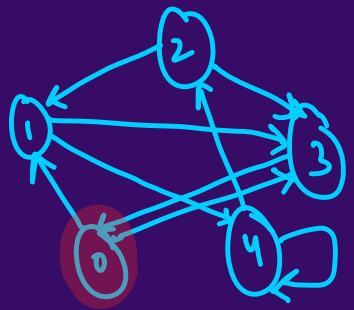
PW SKILLS

Q : Keys and Rooms (BFS)

Can you unlock all rooms or not

'n' rooms → 0 to n-1



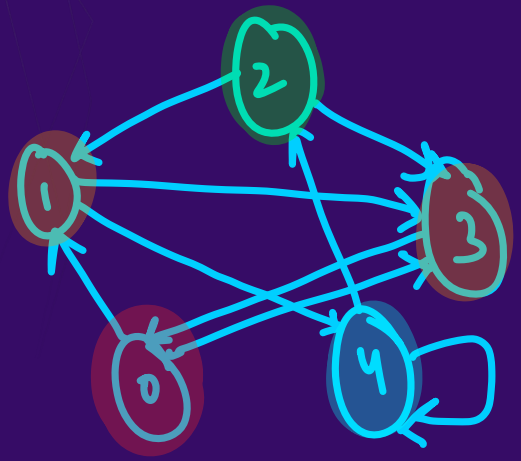


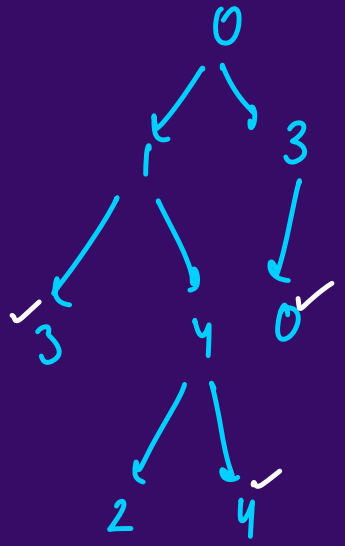
n=5

[Leetcode 841]

Ques:

Q : Keys and Rooms (BFS)





[Leetcode 841]

Ques:

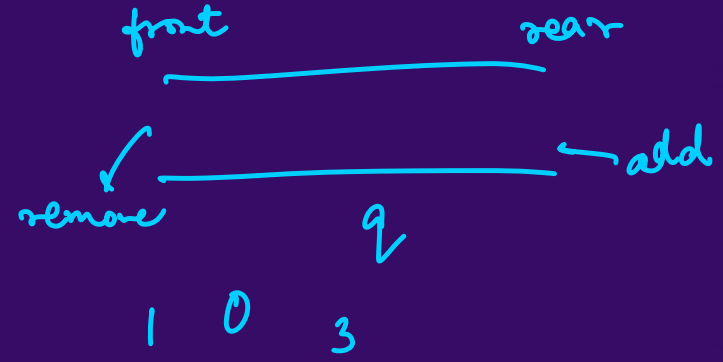
PW SKILLS

Q : Keys and Rooms (BFS)

A.S. = O(n)

T.C. = O(V+E)

adj = { {1,3}, {3,0,1}, {2}, {0} }



visited =

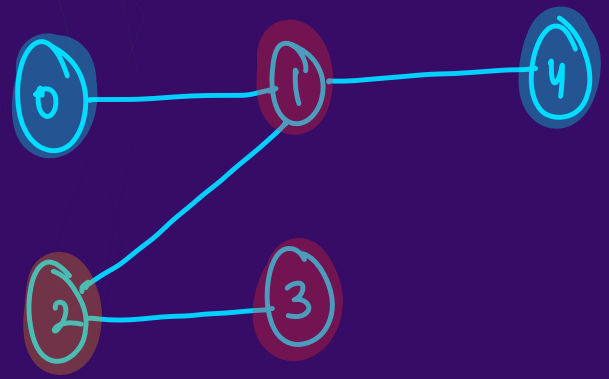
0	1	2	3
T	T	F	T

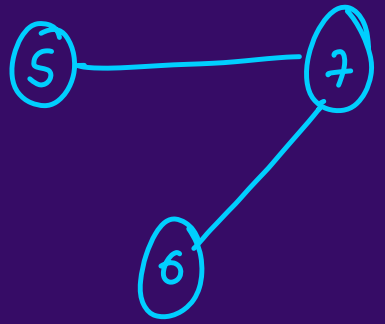
[Leetcode 841]

Ques:

Q : Find if Path Exists in Graph (BFS)

BFS
↑
(start, end)





vis =

0	1	2	3	4	5	6
T	T	T	T	T	F	F

st end

4 → 3 yes

(2) → (5) No

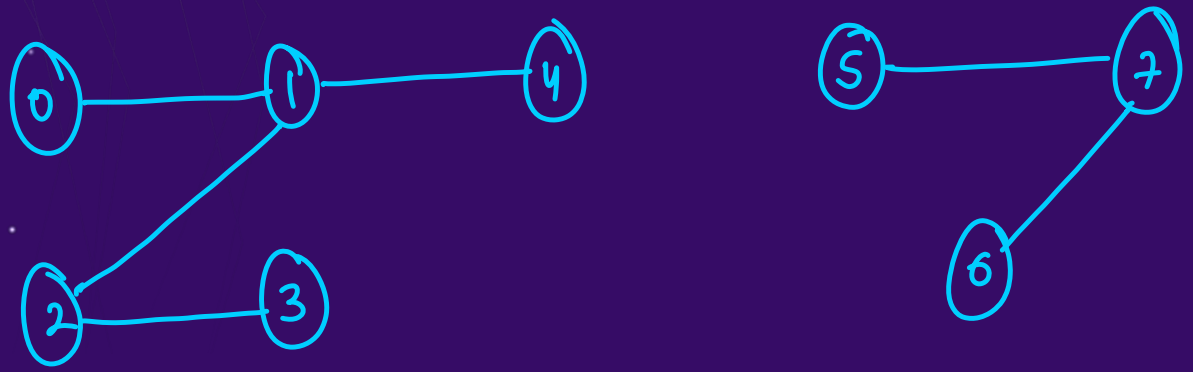
[Leetcode 1971]

Ques:

A.S. = O(V+2E) or O(n+2E)

T.C. = O(n+2E)

Q : Find if Path Exists in Graph (BFS)



0 : 1
1 : 0, 4, 2
2 : 1, 3
3 : 2
4 : 1
5 : 7
6 : 7
7 : 5, 6

edges = { {0,1}, {4,1}, {1,2}, {2,3}, {7,5}, {6,7} }

adj = { {3}, {3}, {3}, {3}, {3}, {3}, {3}, {3} } (0 1 2 3 4 5 6 7)

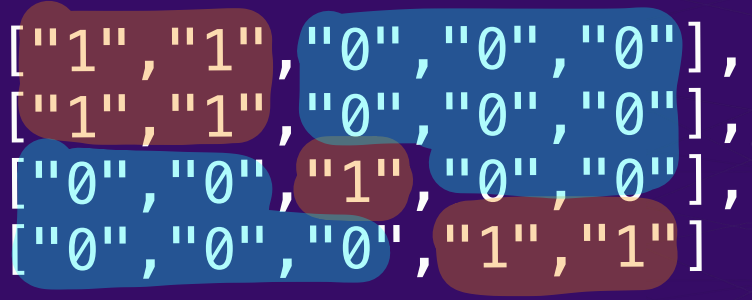
[Leetcode 1971]

Ques:

Connected Components

Q : Number of Islands (BFS)

ans = 1.



ans = 3

[Leetcode 200]

Ques:

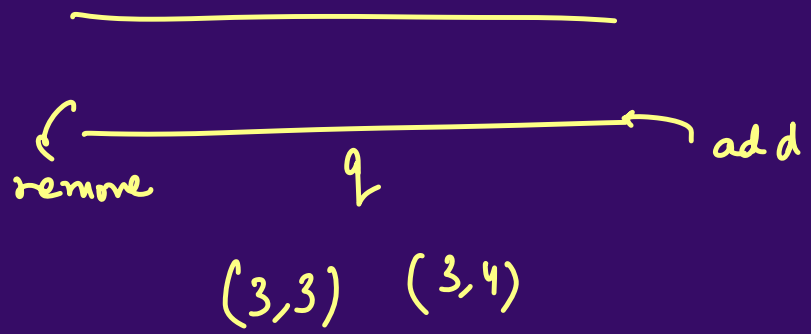
Q : Number of Islands (BFS)

```
    0    1    2    3    4
0 ["1","1","0","0","0"],
1 ["1","1","0","0","0"],
2 ["0","0","1","0","0"],
3 ["0","0","0","1","1"]
```

T	T	F	F	F
T	T	F	F	F
F	F	T	F	F
F	F	F	T	T

vis

Count = 1 2 3



[Leetcode 200]

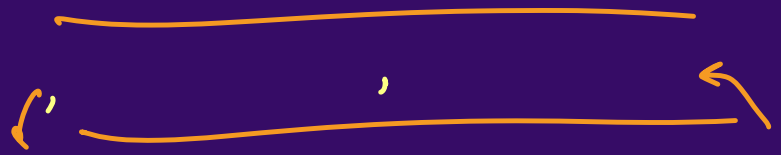
Ques:

Q : Number of Islands (BFS)

```
    0   1   2   3   4
0 ["1","1","1","1","0"],
1 ["1","1","0","1","0"],
2 ["1","1","0","0","0"],
3 ["0","0","0","0","0"]
```

T	T	T	T	F
T	T	F	T	F
T	T	F	F	F
F	F	F	F	F

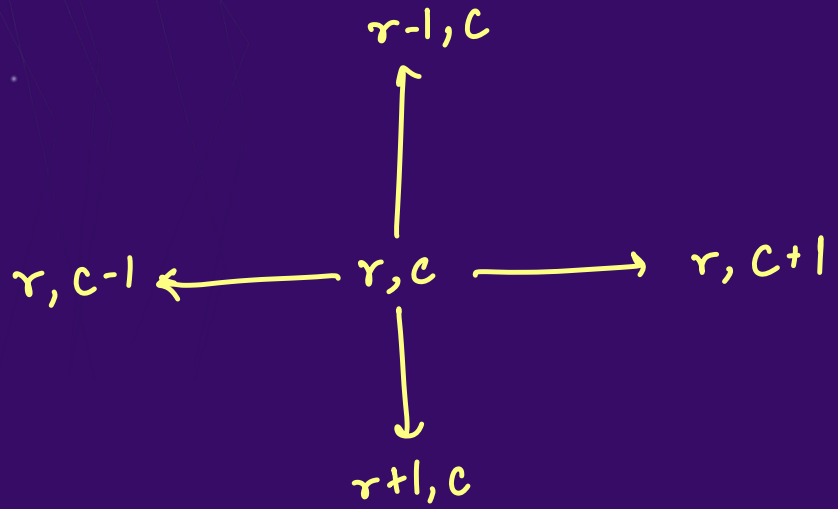
vis



[Leetcode 200]

Ques:

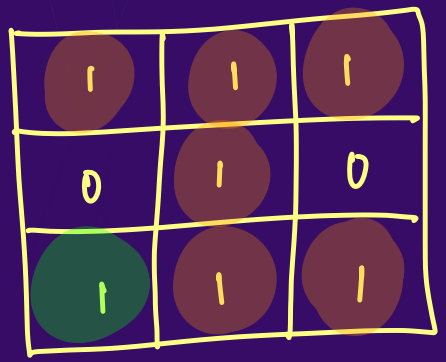
Q : Number of Islands (BFS)

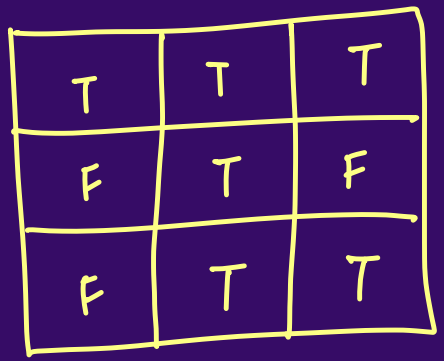


[Leetcode 200]

Ques:

Q : Number of Islands (BFS)





vis

A.S = O(m*n)

T.C. = O(m*n)

you have to apply BFS for all directions

top, bottom, left right

[Leetcode 200]

Depth First Search → We keep traversing down

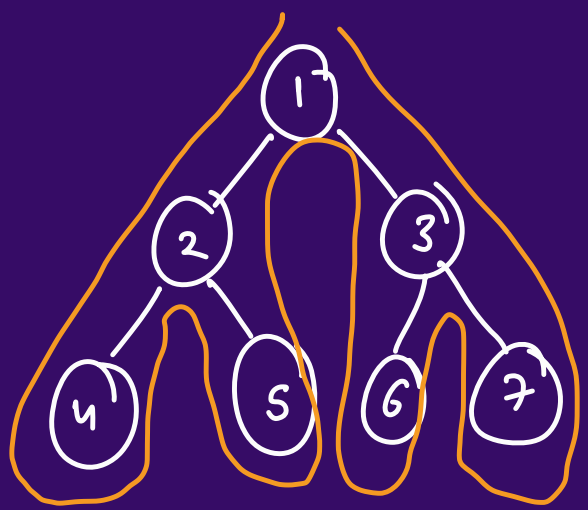
PW SKILLS

Binary Tree → Pre, In, Post

N L R

L N R

L R N

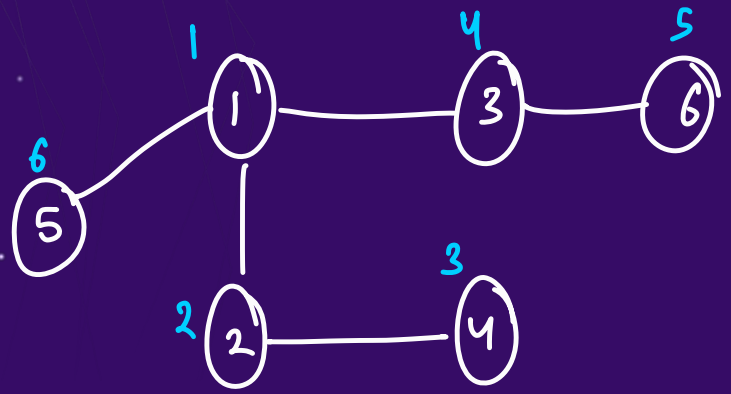


Euler tour

Ques:

Q : Number of Provinces (DFS)

Starting pt.



1 → 2, 3, 5

2 → 1, 4

3 → 1, 6

4 → 2

5 → 1

6 → 3

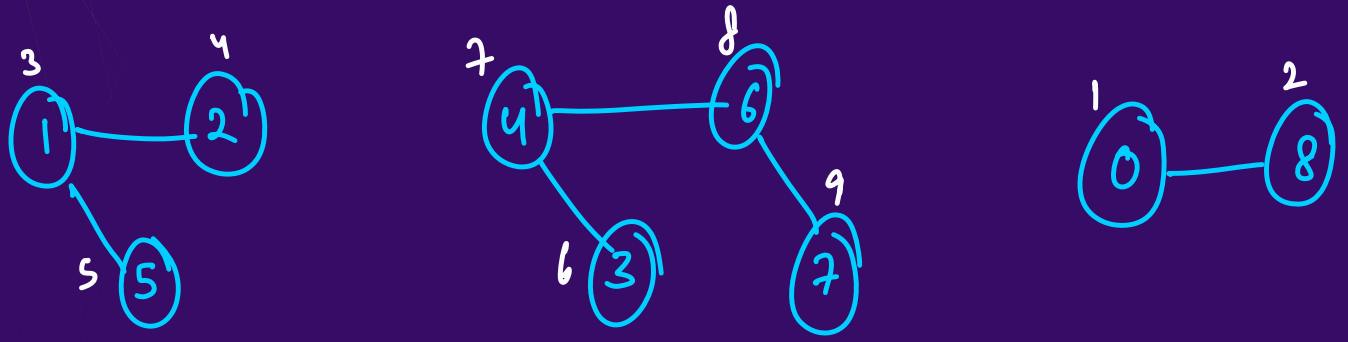
visit =

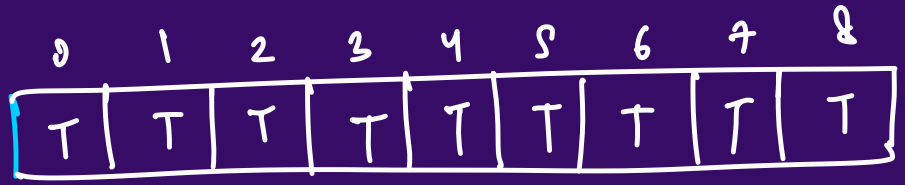
1	2	3	4	5	6
T	T	T	T	T	T

[Leetcode 547]

Ques:

Q : Number of Provinces (DFS)





[Leetcode 547]

Ques:

Q : Number of Islands (DFS)

```
    0    1    2    3    4
0 ["1","1","0","0","0"],
1 ["1","1","0","0","0"],
2 ["0","0","1","0","0"],
3 ["0","0","0","1","1"]
```

T	T	F	F	F
T	T	F	F	F
F	F	F	F	F
F	F	F	F	F

vis

L T R B

[Leetcode 200]

Question

~~Homework:~~

PW SKILLS

Q : Keys and Rooms (DFS)

↓

Logic → Simple Recursion

[Leetcode 841]

Homework:

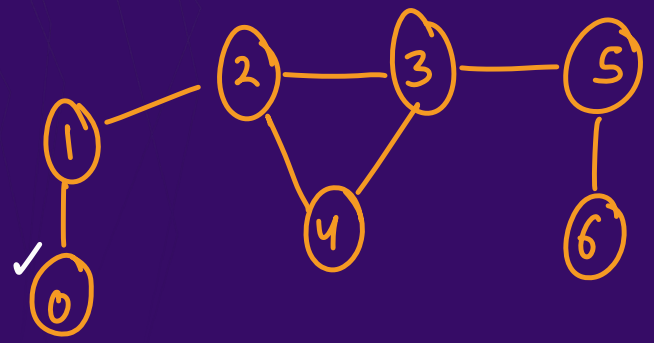
Q : Find if Path Exists in Graph (DFS)

[Leetcode 1971]

Cycle Detection

In Undirected Graph (BFS)

queue < node, parent >



0 → 1

1 → 0, 2

2 → 3, 4, 1

3 → 2, 4, 5

4 → 2, 3

5 → 3, 6

6 → 5

visited =

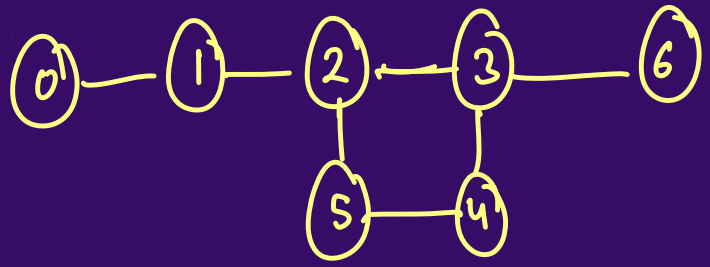
0	1	2	3	4	5	6
T	T	T	T	T		

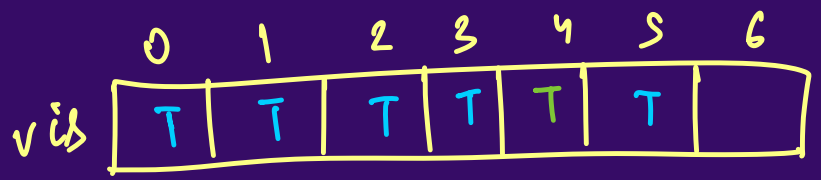
cycle detected

Cycle Detection

In Undirected Graph (DFS)

S.C. = O(V) / O(n)
T.C. = O(V+2E)





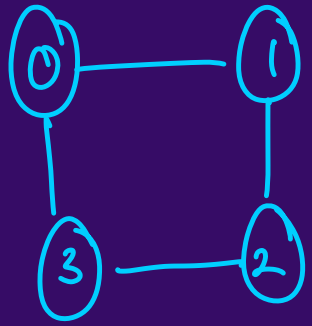
dfs(0,-1)
↓
dfs(1,0)
↓
dfs(2,1)
dfs(3,2)
dfs(5,2)
dfs(4,3)
dfs(6,3)
↓
dfs(5,4)

return true

Ques:

PW SKILLS

Q : Is Graph Bipartite



0 → 1, 3

1 → 0, 2

2 → 1, 3

3 → 0, 2

True

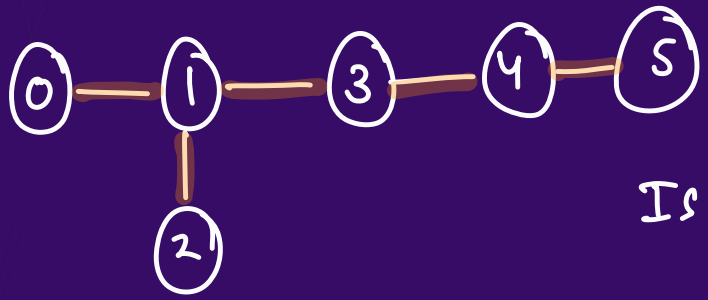
Set A	Set B
0, 2	1, 3

[Leetcode 785]

Ques:

PW SKILLS

Q : Is Graph Bipartite



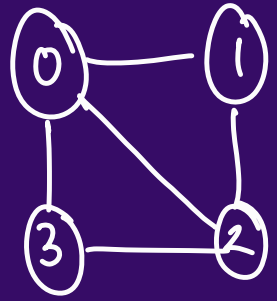
Is Bipartite

set A	set B
0, 2, 3, 5	1, 4

[Leetcode 785]

Ques:

Q : Is Graph Bipartite



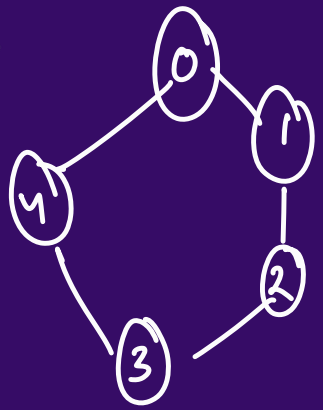
False

set A	set B
1, 3	0

[Leetcode 785]

Ques:

Q : Is Graph Bipartite



False

Set A

0, 2

Set B

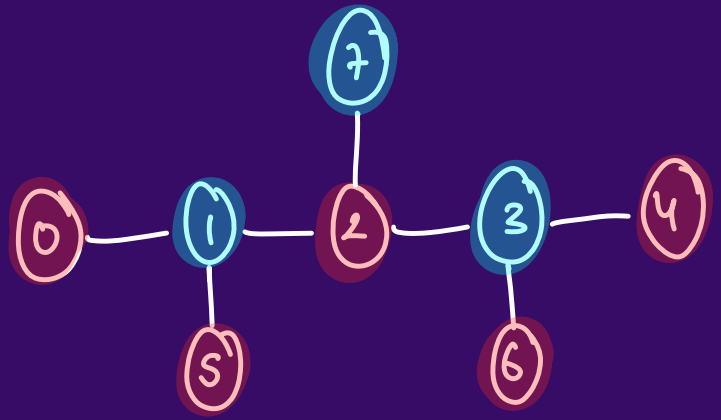
1, 4,

[Leetcode 785]

Ques:

PW SKILLS

Q : Is Graph Bipartite → When we can color each node of the graph with either red or blue & adjacent nodes must have different color.



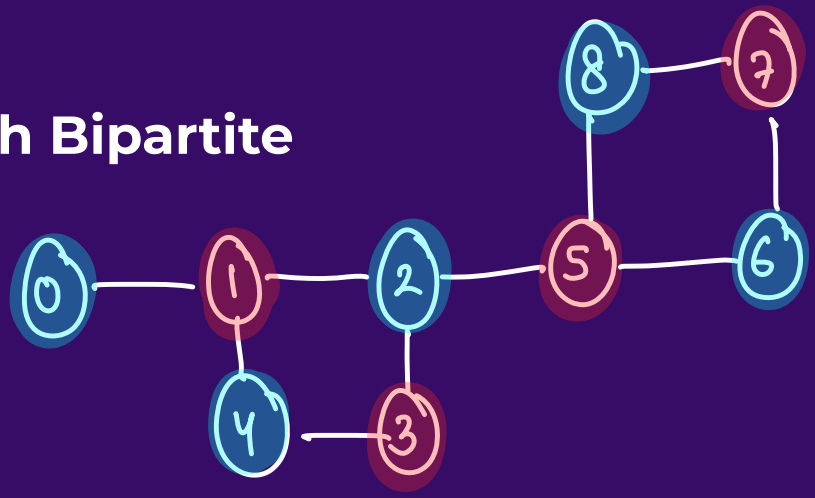
Any acyclic graph is always bipartite

Any cyclic graph with even cyclic length is bipartite

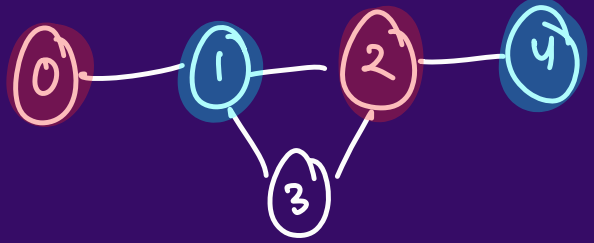
[Leetcode 785]

Ques:

Q : Is Graph Bipartite



yes



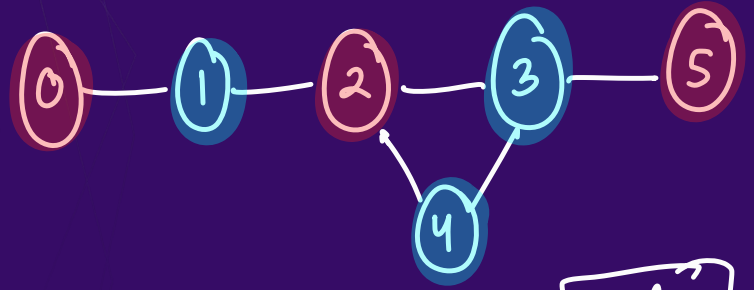
False

[Leetcode 785]

Red = 0
Blue = 1

Ques:

Q : Is Graph Bipartite



False

0 → 1
1 → 0, 2
2 → 1, 3, 4
3 → 2, 5, 4
4 → 2, 3
5 → 3

vis =

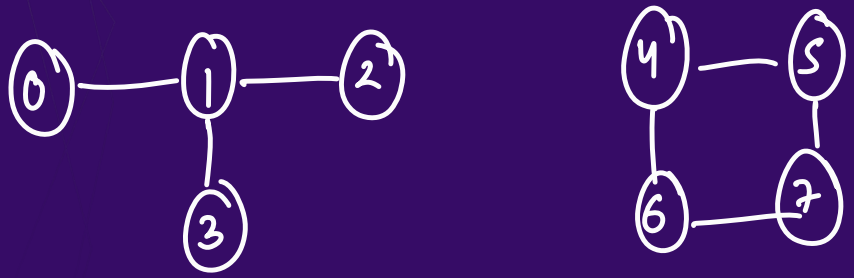
0	1	2	3	4	5
0	1	0	1	1	0

3

[Leetcode 785]

Ques:

Q : Is Graph Bipartite



[Leetcode 785]

Ques:

Q : Is Graph Bipartite

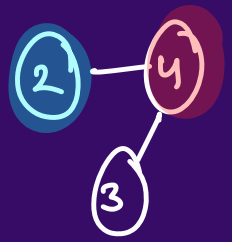
0 → 1

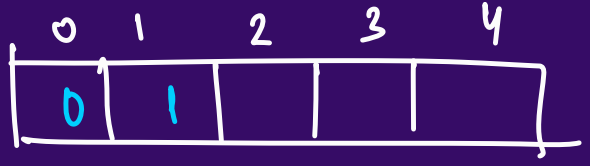
1 → 0

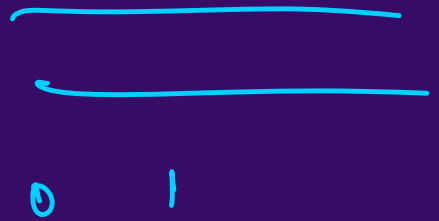
2 → 4

3 → 4

4 → 2, 3





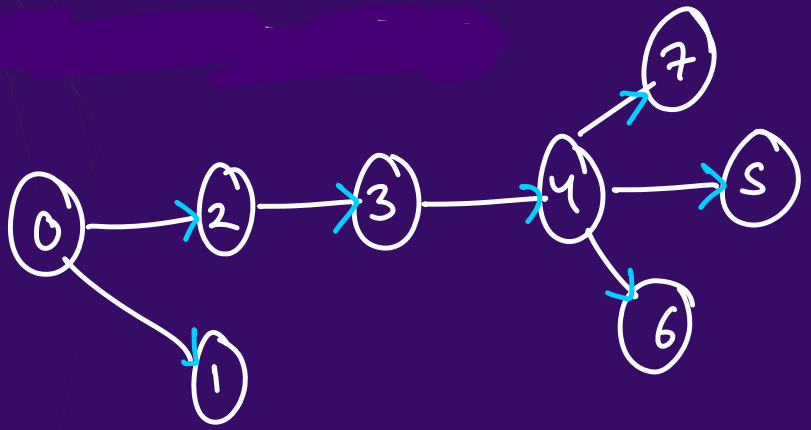


[Leetcode 785]

Topological Sort

BFS → DAG

directed acyclic graph



0 1 2 3 4 5 6 7

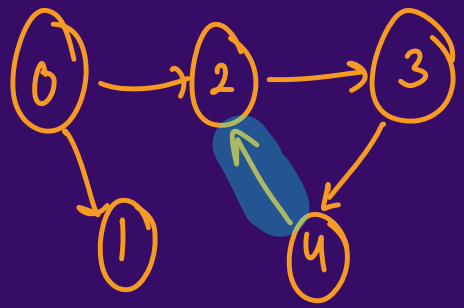
0 2 1 3 4 7 5 6

0 2 3 4 5 6 7 1

Ordering of all nodes of the graph such that if a & b are nodes & a → b then in the ordering a will always come before b

Topological Sort

if there is an edge b/w a & b then a should appear before b



0 1 2 3 4 ✗✗

Cyclic graphs me topological sort nahi hoga

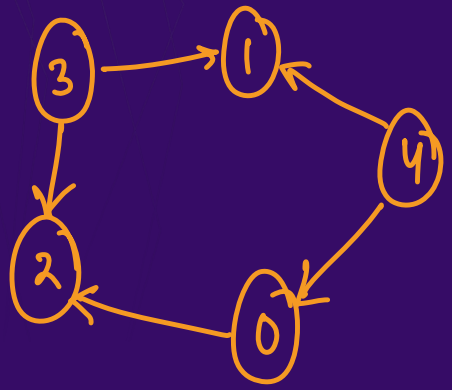
DAG

Topological Sort

3 4 1 0 2

PW SKILLS

DFS



vis =

0	1	2	3	4
T	T	T	T	T

ans = { 2 , 0 , 1 , 3 , 4 } → reverse

4	3	1	0	2

adj

0 → 2

1 →

2 →

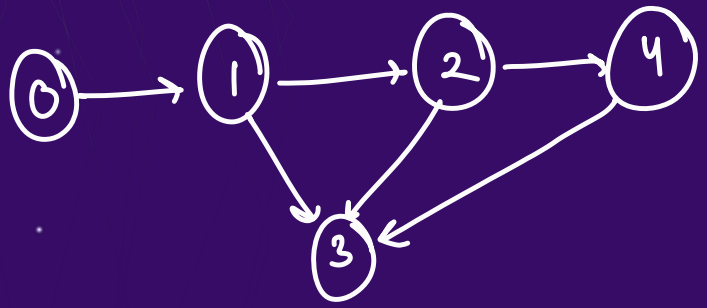
3 → 2, 1

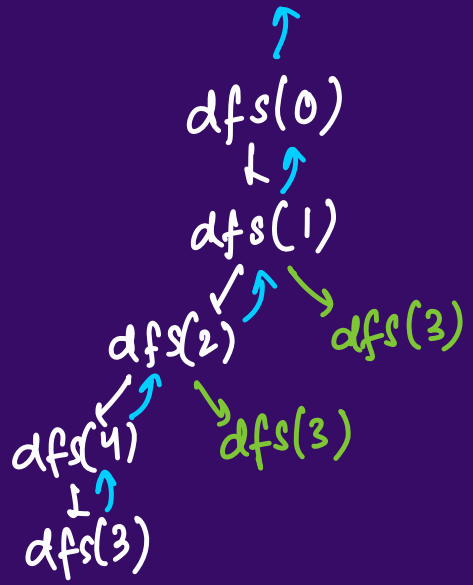
4 → 1, 0

Topological Sort

PW SKILLS

DFS





adj List

0 → 1
1 → 2, 3
2 → 4, 3
3 →
4 → 3

visited =

0	1	2	3	4
T	T	T	T	T

ans = { 3, 4, 2, 1, 0 } rev

0, 1, 2, 4, 3

Topological Sort

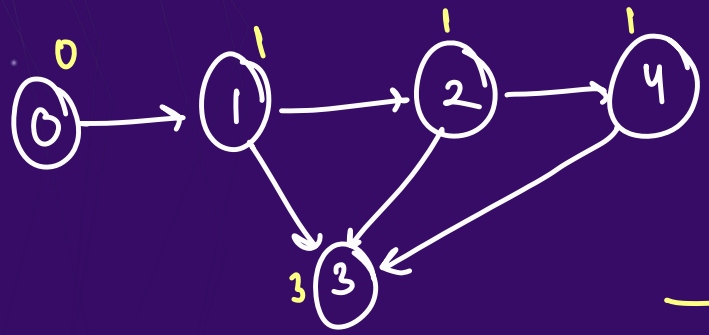
```
for(i=0 to n-1){
    if(!vis[i]) dfs(i)
}

void dfs(i){
    vis(i) = true
    for(int ele : adj[i]){
        dfs(ele)
    }
    ans.add(i)
}
```

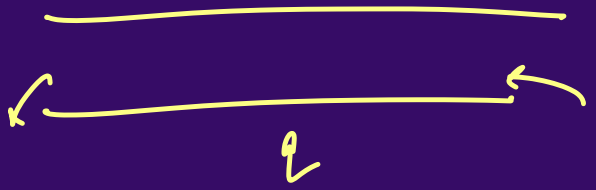
Topological Sort → 0 1 2 4 3

PW SKILLS

BFS (Kahn's Algorithm)



indegree & outdegree



adj. List

0 → 1
1 → 2, 3
2 → 3, 4
3 →
4 → 3

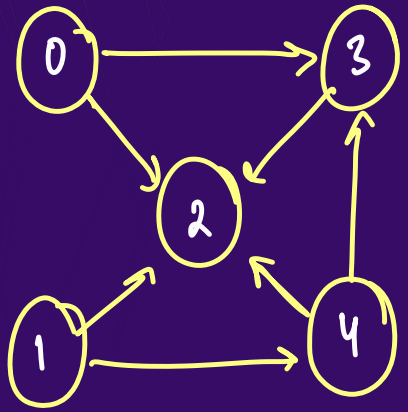
in =

0	1	2	3	4
0	0	0	0	0

ans = { 0 1 2 4 3 }

Topological Sort

BFS (Kahn's Algorithm)



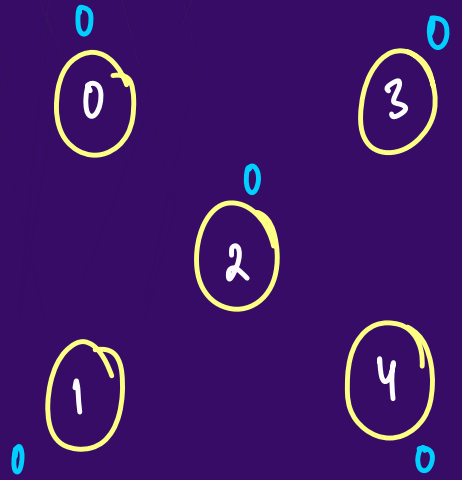
0 1 4 3 2

or

1 4 0 3 2

Topological Sort

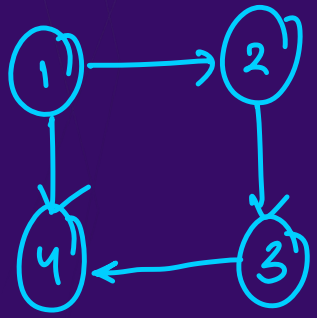
BFS (Kahn's Algorithm)



ans = { 1 0 4 3 2 }

Cycle Detection

Directed Graph (BFS)



There is a cycle (BFS) says so.

False

(1,-1) (4,1) (3,2)

(2,1)

1	2	3	4
T	T	T	T

Cycle Detection

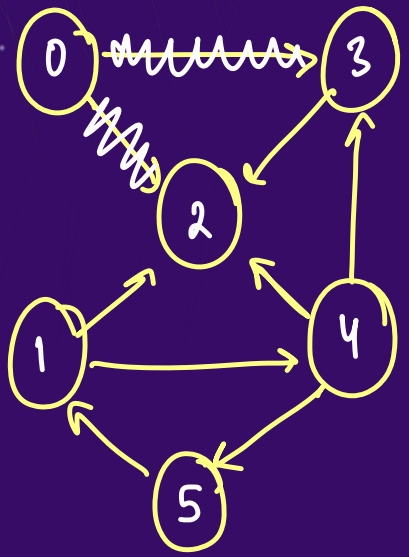
PW SKILLS

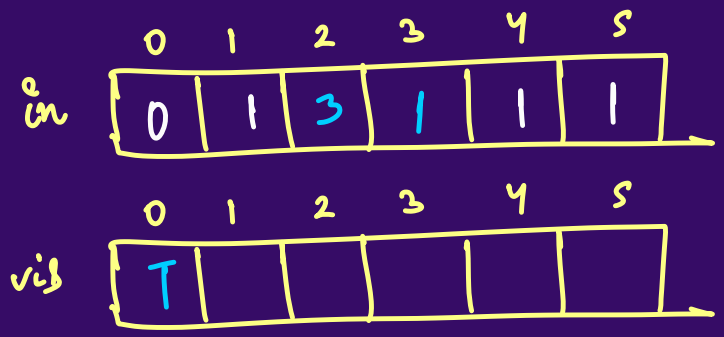
Directed Graph (BFS)

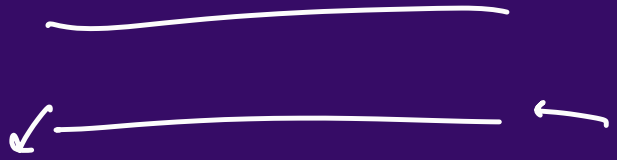
- In a DAG, when we apply topological sort → BFS (Kah'n) we visit 'all' the nodes
- So if we have a Directed Cyclic graph & we apply Kah'n algorithm then something unusual happens

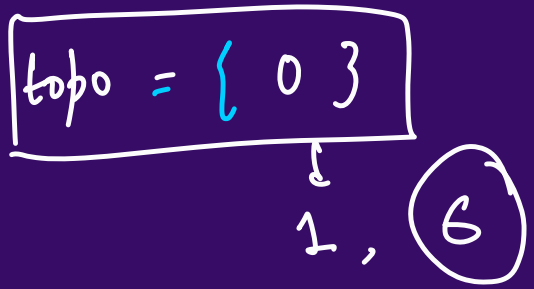
Cycle Detection

Directed Graph (BFS)



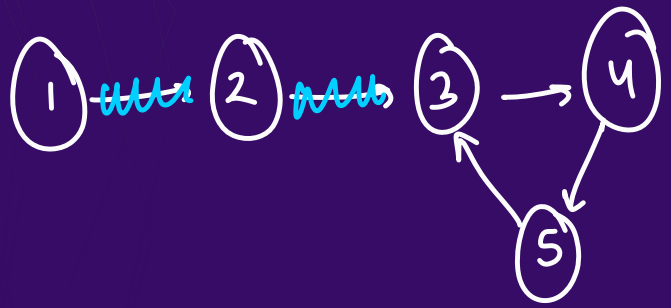


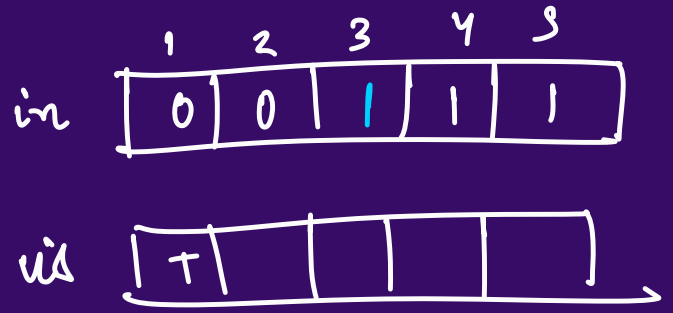




Cycle Detection

Directed Graph (BFS)



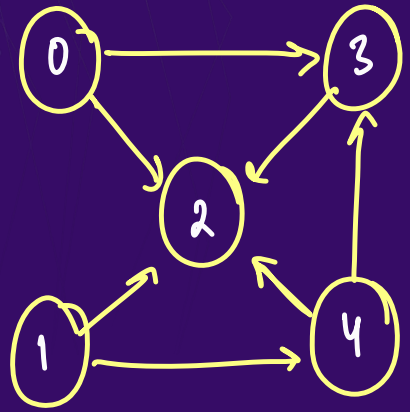


ans = { 1 2 }

size = 2

Cycle Detection

Directed Graph (BFS)



in

0	1	2	3	4
0	0	0	0	0

topo = { 0 1 4 3 2

Ques:

PW SKILLS

Q : Course Schedule

pre = { {3, 2}, {1, 0}, {2, 1}, {4, 3}, {0, 3} }

In this directed graph, if it is cyclic then → false
else → true

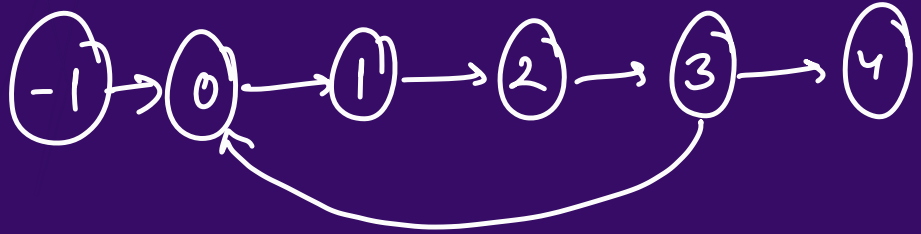
(3) 0 1 2 (3) 4

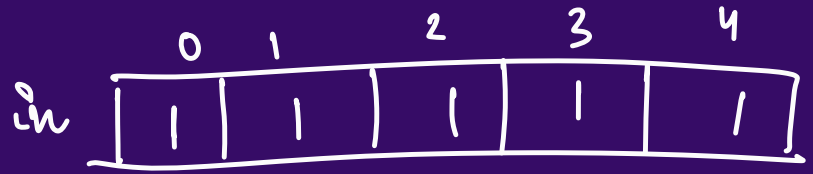
[Leetcode 207]

Ques:

Q : Course Schedule

pre = { {3, 2}, {1, 0}, {2, 1}, {4, 3}, {0, 3} }





AL

0 → 1

1 → 2

2 → 3

3 → 0, 4

4 → ∝

[Leetcode 207]

Ques:

Q : Course Schedule II

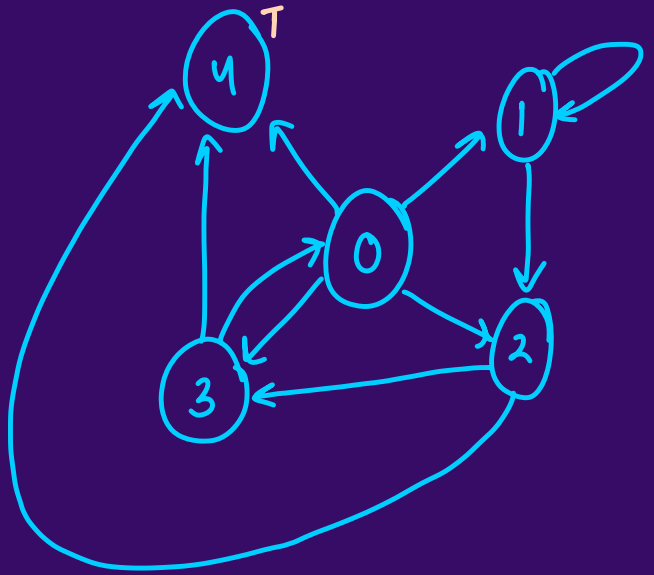
[Leetcode 210]

Ques:

Q : Find eventual safe states

0 1 2 3 4

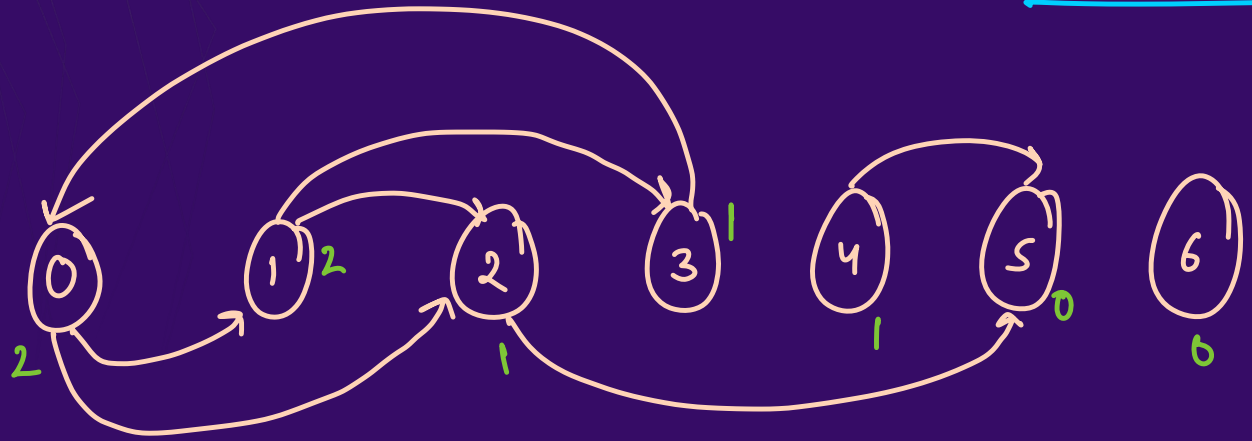
{ {1,2,3,4} , {1,2}, {3,4}, {0,4}, {} }

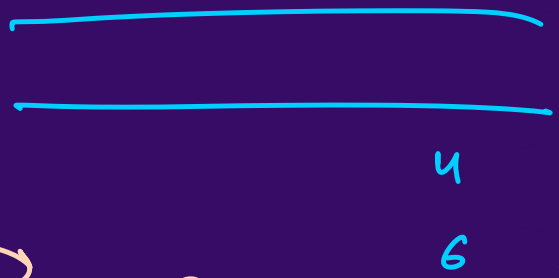


[Leetcode 802]

Ques:

Q : Find eventual safe states





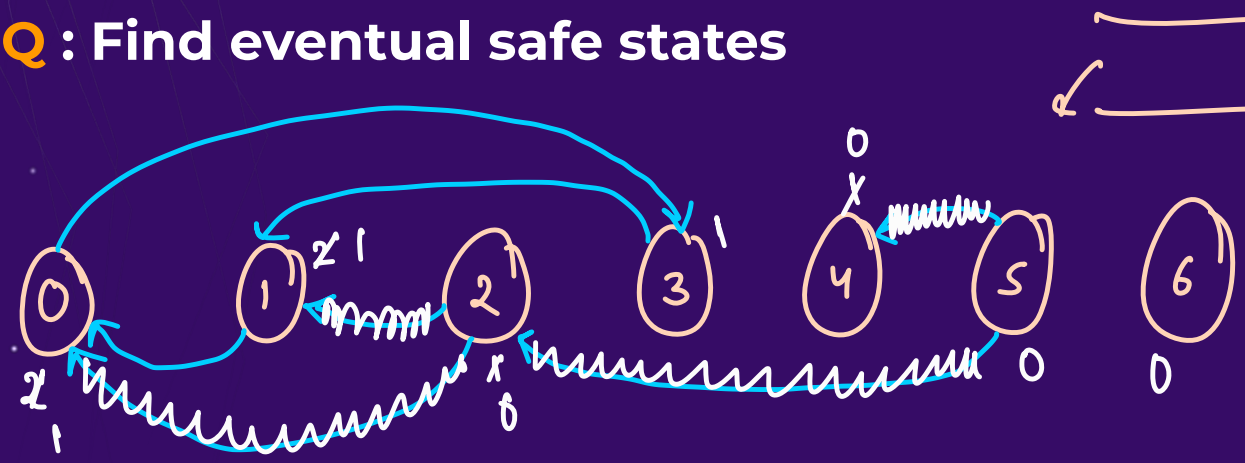
Indegree → incoming edges

terminal node → no outgoing edges → outdegree → 0

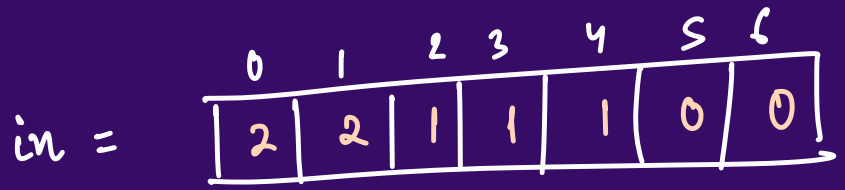
[Leetcode 802]

Ques: Find all nodes which are not part of a cycle

Q : Find eventual safe states



ans = {5, 6, 4, 2}



[Leetcode 802]

HW → GFG

Ques: English → a, b, c, d . . . . z

PW SKILLS

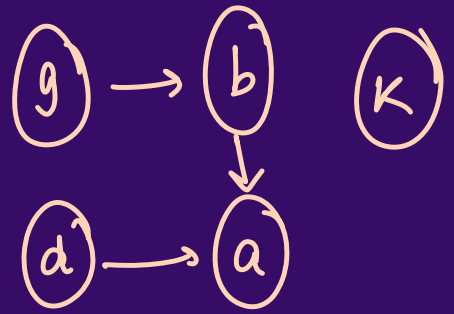
Q : Alien Dictionary → Topological Sort

Alien language → ?

arr = { gag, bda, bak, adg }

g, b, d, a, k

or



Ques:

Q : Alien Dictionary

26 size graph

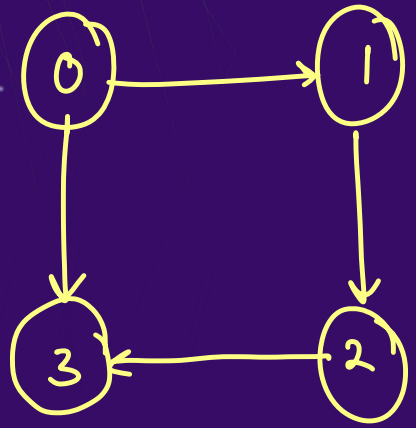
a b c d z

adj = { b, g, c , a, d, h , — , — . . . }

BFS → cycle → Kahn's

Ques:

Q : Cycle Detection (DFS) (Directed graph)



dfs(0, -1)

~~dfs(1, 0)~~

~~dfs(2, 1)~~

~~dfs(3, 2)~~

dfs(3, 0)

detected.

but cycle was not there.

vis =

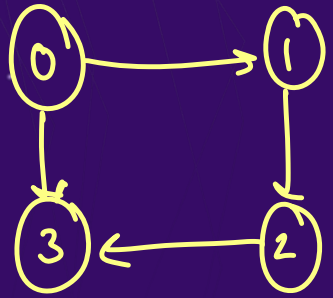
0	1	2	3
T	T	T	T

0 → 1 → 2 → 3

& 0 → 3 are diff. paths

Ques:

Q : Cycle Detection (DFS)

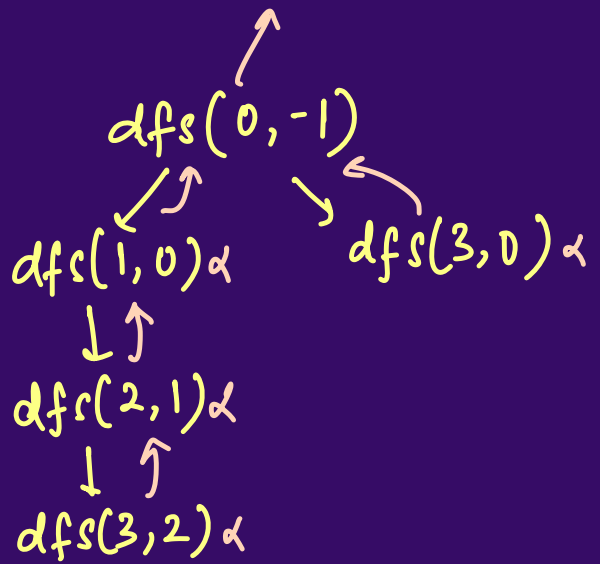


vis =

0	1	2	3
T	T	T	T

path =

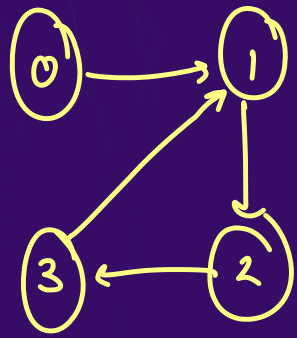
0	1	2	3



Ques: we dont need to pass the parent

PW SKILLS

Q : Cycle Detection (DFS)



vis =

0	1	2	3
T	T	T	T

path =

0	1	2	3
T	T	T	T

dfs(0,-1)

↓

dfs(1,0)

↓

dfs(2,1)

↓

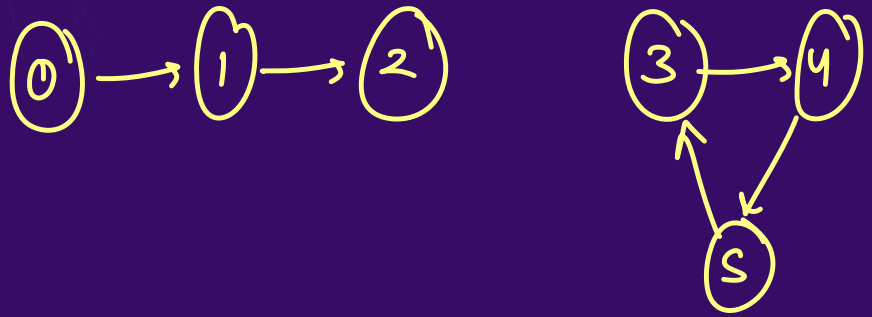
dfs(3,2)

↓

dfs(1,3) → but 1 is in same path

Ques:

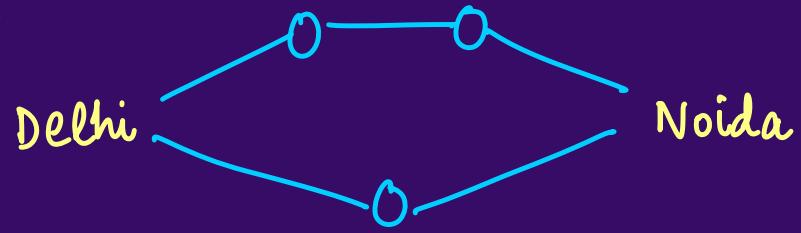
Q : Cycle Detection (DFS)



Dijkstra's Algorithm

PW SKILLS

Shortest Path



Locations
↓
Cost, Time, Distance
Mix

✓ Least Cost → Via Metro → 2 hrs 100 Rupees

✓ Least Time → Via Cab → 1 hr 900 Rupees

Avg. Cost → Thodi si Cab, thodi si metro 1.5 hrs 300 rupees

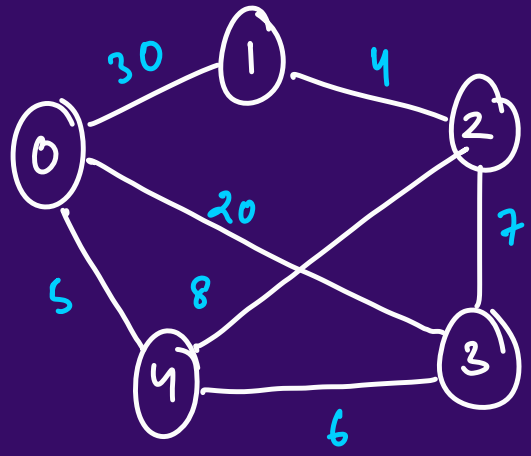
Dijkstra's Algorithm

Undirected, Weighted

PW SKILLS

Shortest Path

we have to go from 0 to all other in min cost



weight → cost / distance

nodes → locations

0	1	2	3	4
0	17	13	11	5

dist =

1

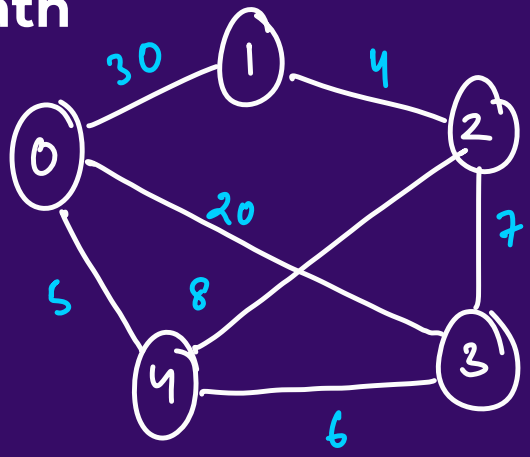
0 se leke

Dijkstra's Algorithm

BFS → Queue ∝
→ PQ. [Heaps]

PW SKILLS

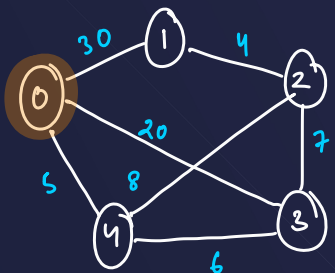
Shortest Path



adj = 0 → (1,30), (4,5), (3,20)
1 → (0,30), (2,4)
2 → (1,4), (3,7), (4,8)
3 → (0,20), (2,7), (4,6)
4 → (0,5), (2,8), (3,6)

List < List < Pair >> adj;

Queue. BFS → Thoda sa alag



adj = 0 → (1,30), (4,5), (3,20)
1 → (0,30), (2,4)
2 → (1,4), (3,7), (4,8)
3 → (0,20), (2,7), (4,6)
4 → (0,5), (2,8), (3,6)

dis =

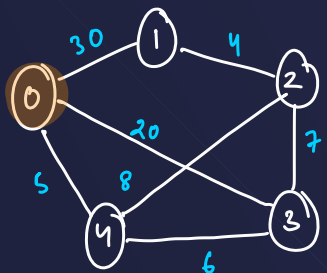
0	1	2	3	4
0	17	13	11	5

queue <node, distance>

nodes = n
edges = n^2

T.C. = $O(n^2)$

Heap.



adj = 0 → (1,30), (4,5), (3,20)
1 → (0,30), (2,4)
2 → (1,4), (3,7), (4,8)
3 → (0,20), (2,7), (4,6)
4 → (0,5), (2,8), (3,6)

cost =

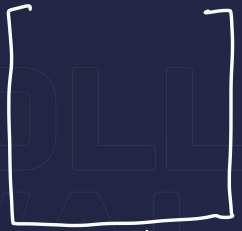
0	1	2	3	4
0	17	13	11	5

(0,30) 60

(1,30) → (2,4)

top

p

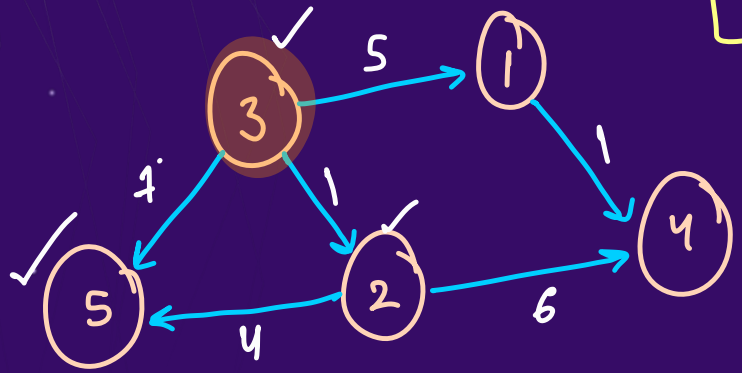


```
pq.add(new pair(0,0))
while(pq.size()>0){
    pair top = pq.remove()
    for(pair p: adj.get(node)){
        totaldist = top.dist + p.dist
        if( dist[p.node] > totaldist){
            dist[p.node] = totaldist
            pq.add(new pair(p.node, total))
        }
    }
}
```

Ques:

PW SKILLS

Q : Network Delay Time



k=3

k^{th} tower is transmitting signal. Find the minimum time in which the signal can transmit to all towers.

1	2	3	4	5
5	1	0	6	5

max

at t=0 → 3
t=1 → 3,2
t=2 → 3,2
t=3 → 3,2
t=4 → 3,2

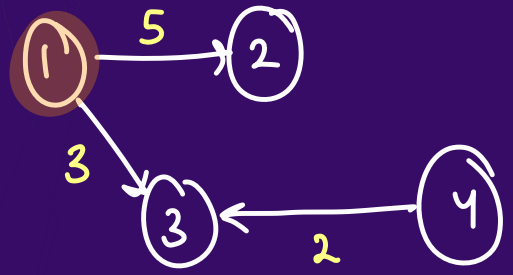
t=5 3,2,5,1
t=6 3,2,5,1,4

ans = 6

[Leetcode 743]

Ques:

Q : Network Delay Time

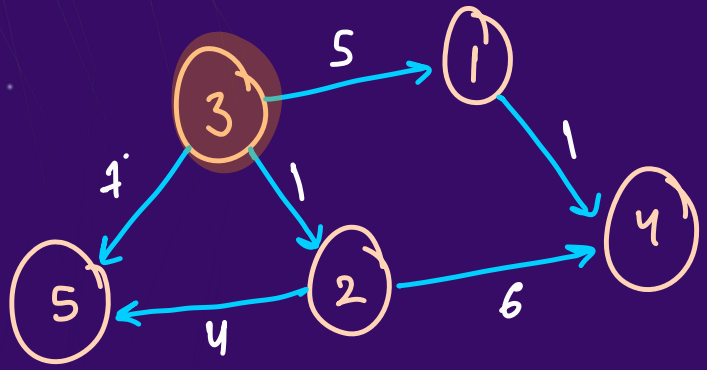


ans = -1

[Leetcode 743]

Ques:

Q : Network Delay Time



adj

node

time

0 :

1 : (4, 1)

2 : (5, 4), (4, 6)

3 : (1, 5), (2, 1), (5, 7)

4 :

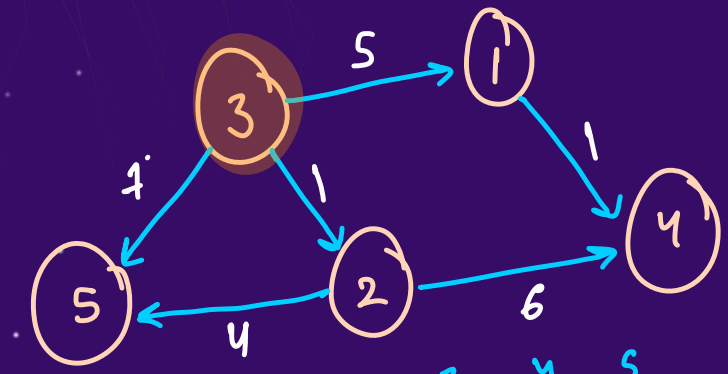
5 :

edges = {{3, 1, 5}, {3, 2, 1}, {3, 5, 7}, {2, 5, 4}, {2, 4, 6}, {1, 4, 1}}

[Leetcode 743]

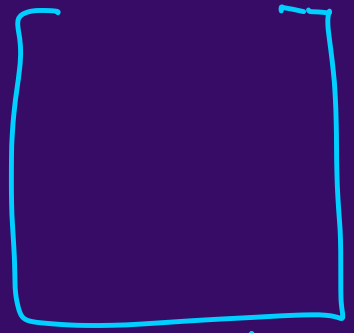
Ques:

Q : Network Delay Time



ans =

0	1	2	3	4	5
X	5	1	0	6	5



minheap

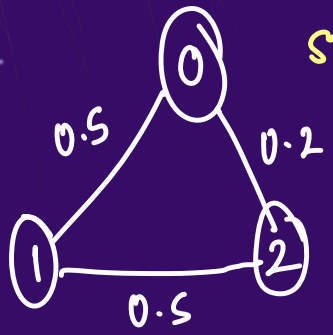
<node, time>

[Leetcode 743]

Ques:

0 <= p <= 1

Q : Path with Maximum Probability



st = 0, end = 2

n = 3 → 0 to 3-1

edges = { {0,1}, {1,2}, {0,2} }

succ = { 0.5 , 0.5, 0.2 }

adj =

0 : (1, 0.5) (2, 0.2)

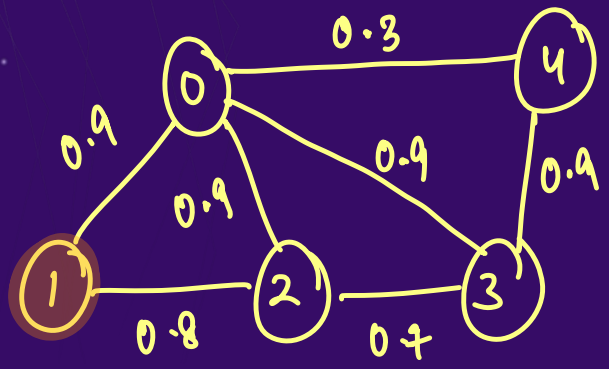
1 : (0, 0.5) (2, 0.5)

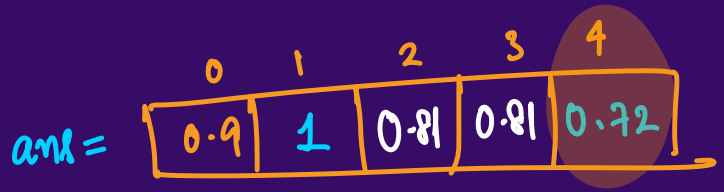
2 : (0, 0.2) (1, 0.5)

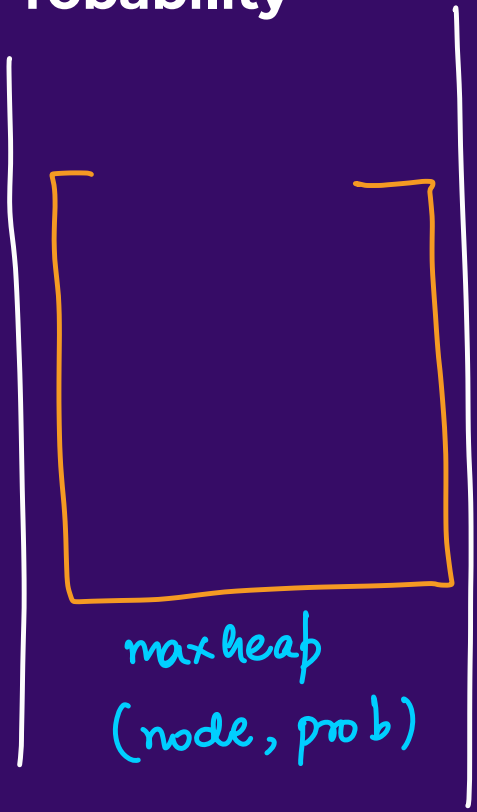
[Leetcode 1514]

Ques: st=1, end=4

Q : Path with Maximum Probability





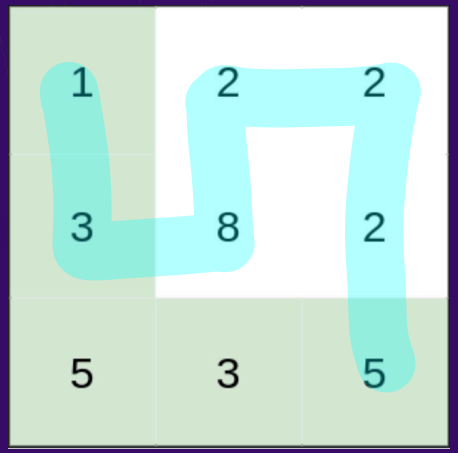


[Leetcode 1514]

Ques:

Q : Path with Minimum Effort

Up, Left, Down, Right



1 - 3 - 5 - 3 - 5

2 2 2 2

→ max = 2 ⇒ effort

1 - 2 - 2 - 2 - 5

1 0 0 3

⇒ max = 3 → effort

2 5 6 0 0 3

effort

[Leetcode 1631]

Minimum Effort

	0	1	2
0	1	2	2
1	3	8	2
2	5	3	5

	0	1	2
0	0	1	1
1	2	5	1
2	2	2	2

ans

(2,2,2)
(2,1,2)
(1,1,5)
(2,0,2)
(2,2,3)
(1,2,1)
(0,2,1)
(1,1,6)
(1,0,2)
(0,1,1)
(0,0,0)

minheap
{row, col, dist}

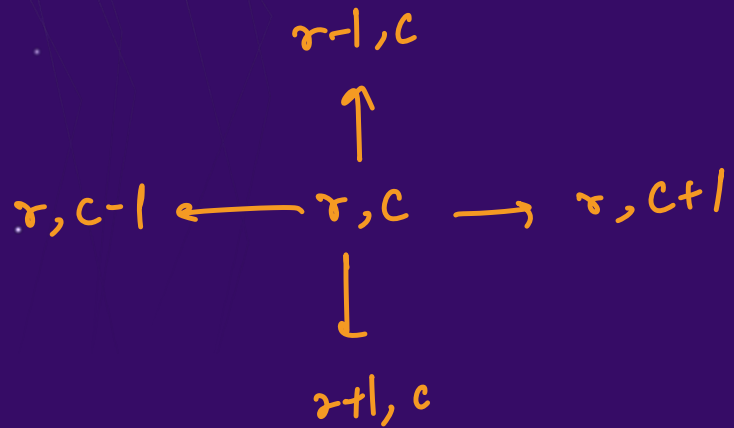
2 → return
(2,2)

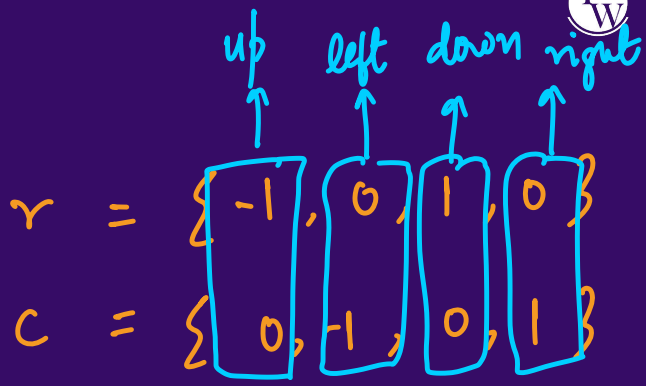
SKILLS

[Leetcode 1631]

Ques:

Q : Path with Minimum Effort





[Leetcode 1631]

Ques:

Q : Path with Minimum Effort

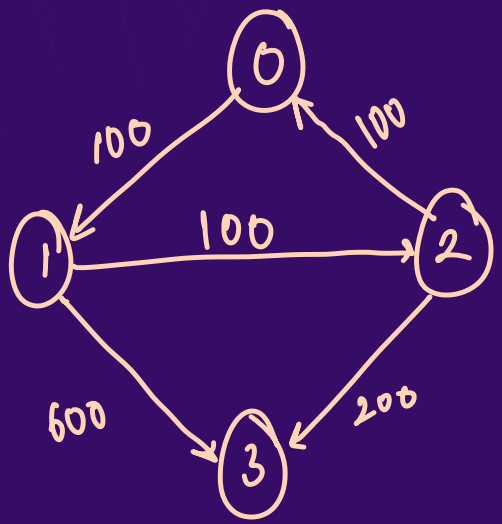
[Leetcode 1631]

Ques:

K+1 stops → including dst

PW SKILLS

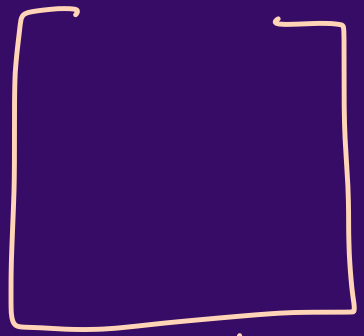
Q : Cheapest flights within K stops



src = 0 , dst = 3 , K = 1

(3, 700, 2)

dst — stops = K+1



minHeap

(node, cost, stops)

ans =

0	1	2	3
0	100	200	700

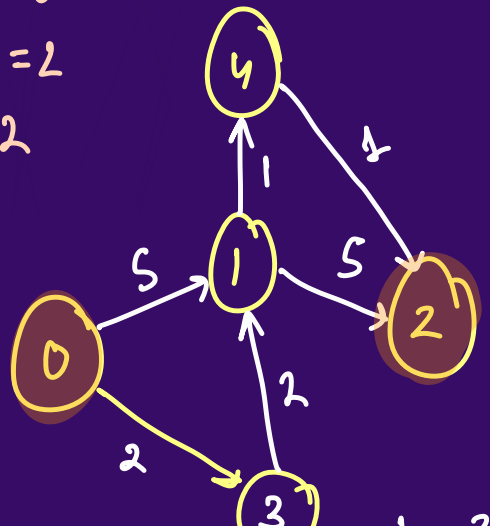
[Leetcode 787]

Ques:

Q : Cheapest flights within K stops

n = 5 flights = { {0,1,5}, {1,2,5}, {0,3,2}, {3,1,2}, {1,4,1}, {4,2,1} }

src = 0
dst = 2
K = 2



7

ans =

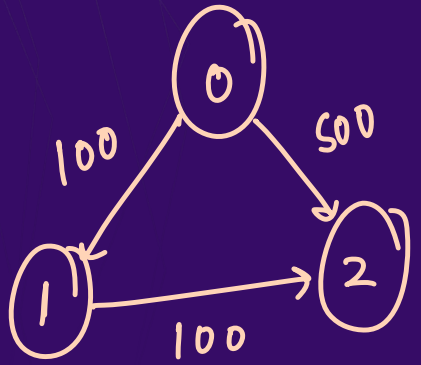
0	1	2	3	4
0	4	7	2	5

~~(4,5,3)~~
~~(2,7,3)~~
~~(1,4,2)~~
~~(4,6,2)~~
~~(2,10,2)~~
~~(3,2,1)~~
~~(1,5,1)~~
~~(0,0,0)~~

[Leetcode 787]

Ques:

Q : Cheapest flights within K stops



src = 0
dst = 2
K = 1

(2, 200, 2) ←

ans =

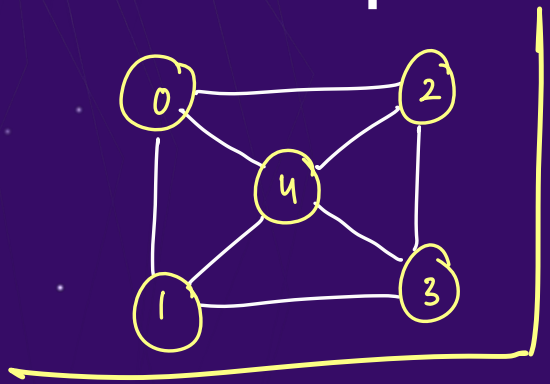
0	1	2
0	100	200

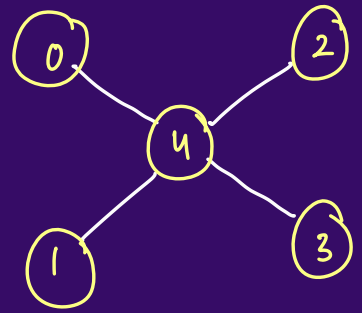
[Leetcode 787]

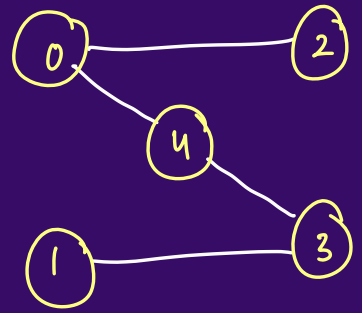
Prim's Algorithm

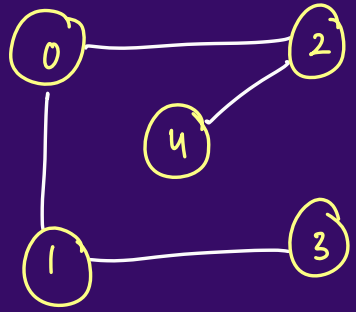
→ Undirected Graph with one component

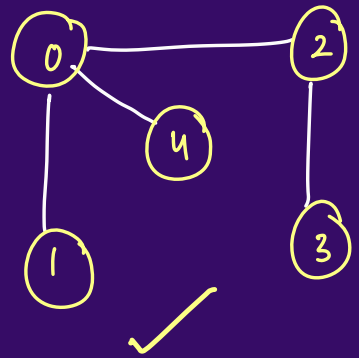
Minimum Spanning Tree

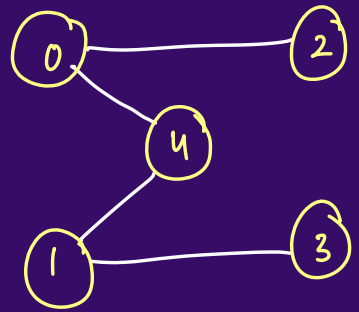






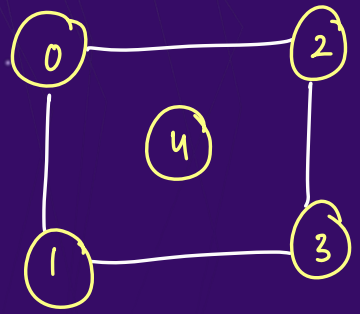




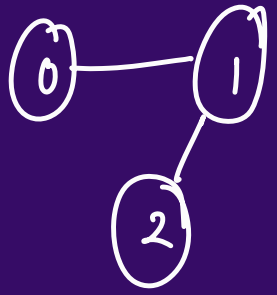


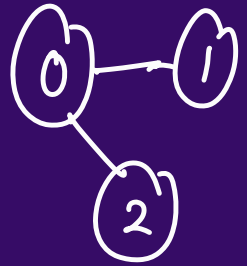
Prim's Algorithm

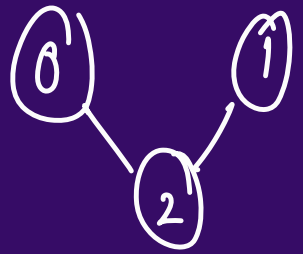
Minimum Spanning Tree



Not a spanning tree





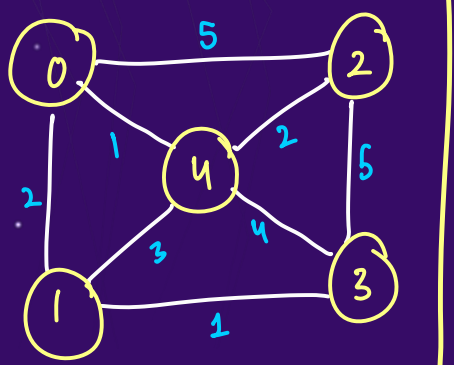


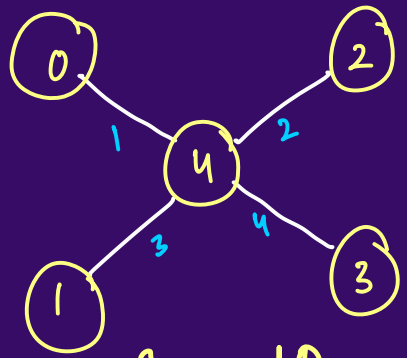
Prim's Algorithm

Minimum Spanning Tree

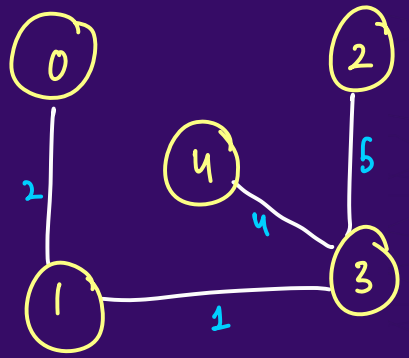
Undirected weighted graph with one connected component.

PW SKILLS

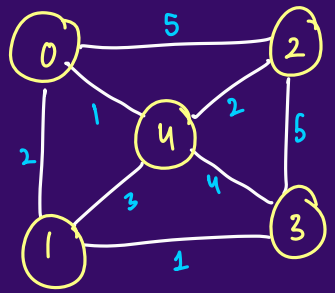


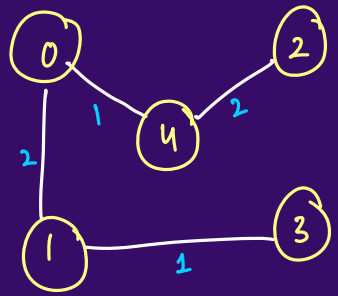


Sum = 10

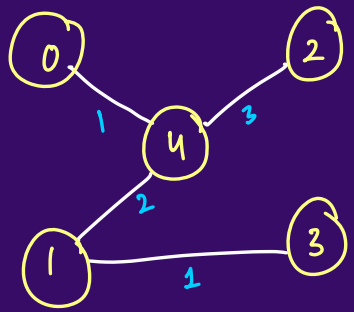


Sum = 12



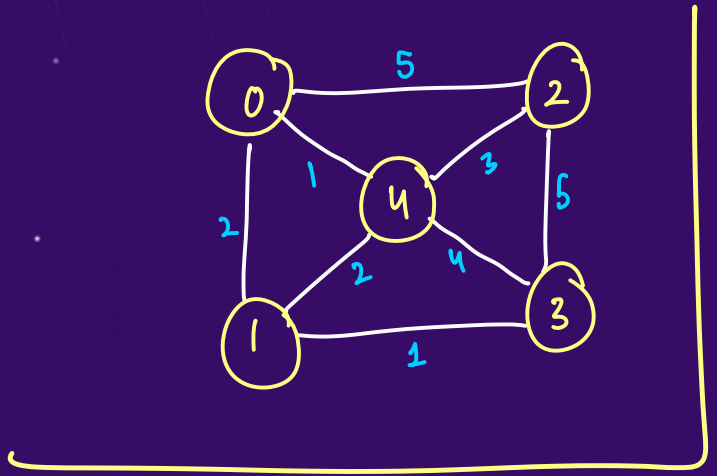


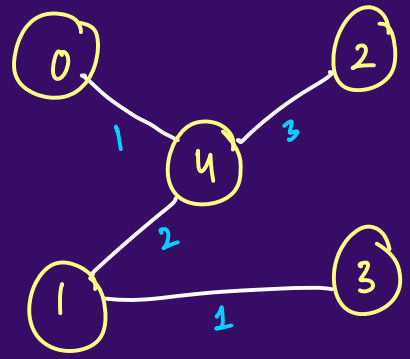
Sum = 6



Prim's Algorithm

Minimum Spanning Tree





0 → (4, 1)

1 → (3, 1), (4, 2)

2 → (4, 3)

3 → (1, 1)

4 → (1, 2), (2, 3)

or

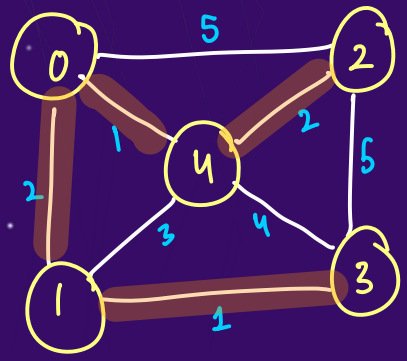
edges =

[(0, 4, 1), (2, 4, 3),
(1, 4, 2) (1, 3, 1)]

Prim's Algorithm

Minimum Spanning Tree

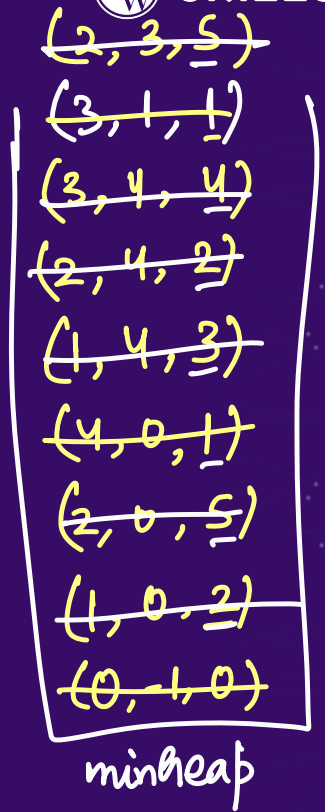
Sum
0 1 3 4 6



vis =

0	1	2	3	4
T	T	T	T	T

MST = { (4,0,1) , (1,0,2) , (3,1,1) , (2,4,2) }

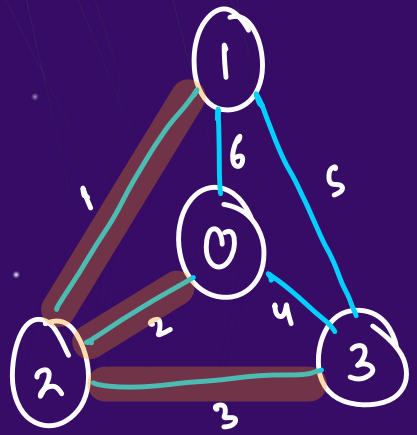


minheap

node, parent, wt

Prim's Algorithm

Minimum Spanning Tree



MST = {(2, 1, 1), (0, 2, 2), (3, 2, 3)}

vis =

0	1	2	3
T	T	T	T

~~(1, 3, 5)~~ SKILLS

~~(1, 3, 5)~~

~~(2, 0, 4)~~

(1, 0, 6)

~~(3, 2, 3)~~

~~(0, 2, 2)~~

(3, 1, 5)

~~(2, 1, 1)~~

(0, 1, 6)

~~(1, -1, 0)~~

minheap

(node, parent, wt)

Ques:

Q : Min Cost to connect all Points

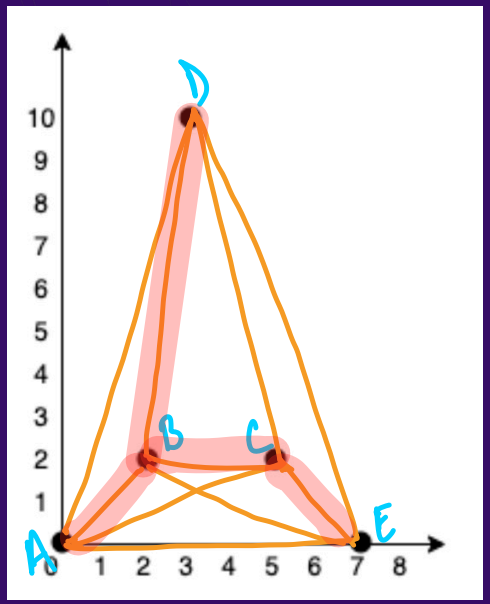
$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$md = |x_1 - x_2| + |y_1 - y_2|$$

[Leetcode 1584]

Ques:

Q : Min Cost to connect all Points



(D, B, 9)

Sum
1
9
4
37
14
20

20

vis =

A	B	C	D	E
T	T	T	T	T

(D, E, 14)
(D, C, 10)
~~(E, C, 4)~~
~~(C, B, 3)~~
~~(D, B, 9)~~
~~(E, B, 7)~~
~~(C, A, 7)~~
~~(E, A, 7)~~
(D, A, 13)
~~(B, A, 4)~~
~~(A, 2, 0)~~

node, parent

[Leetcode 1584]

Bellman-Ford Algorithm

PW SKILLS

Shortest Distance from 'src' node to other nodes in a dist array

Dijkstra → directed, undirected

Bellman → directed

Negative Cycles

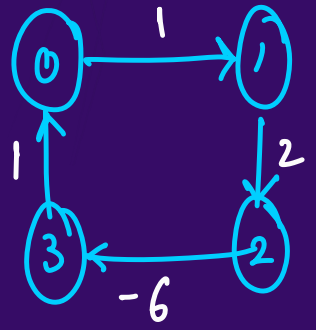
Bellman-Ford Algorithm

Shortest Distance

→ T.C. = $O(V*E)$

A.S. = depends on problem

Negative Cycles



	0	1	2	3
dist	~~-2~~ -40	-1 ~~X~~	~~3~~ 1	~~-3~~ -5

0 to 1

src = 0

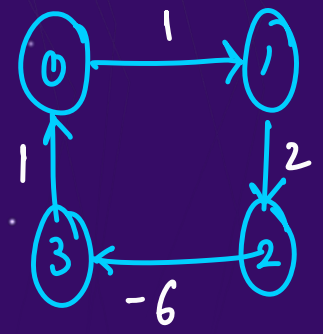
$0 \to 1 \to 2 \to 3 \to 0 \to 1 = 1 + 2 + -6 + 1 + 1$

$= -1$

$0 \to 1 = 1$

Bellman-Ford Algorithm

Shortest Distance



2 $\xrightarrow{-6}$ 3 ∝ ∝ ✓ ✓

1 $\xrightarrow{2}$ 2 ∝ ✓ ✓ ✓

0 $\xrightarrow{1}$ 1 ✓ ✓ ✓ ✓

3 $\xrightarrow{1}$ 0 ∝ ∝ ✓

2, 3, -6

1, 2, 2

0, 1, 1

3, 0, 1

src = 0

	0	1	2	3
dist	-2	1	3	-3

n = 4

'n-1' times edges relaxed, now 1 more time

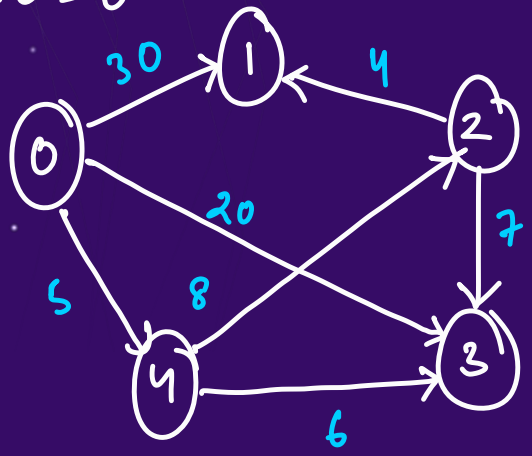
Bellman-Ford Algorithm

Edge → u, v, wt

PW SKILLS

Shortest Distance { Edge-List } n-1

Src = 0



2 → 3 (7)

4 → 2 (8)

0 → 3 (20)

2 → 1 (4)

0 → 1 (30)

4 → 3 (6)

0 → 4 (5)

2, 3, 7

4, 2, 8

0, 3, 20

2, 1, 4

0, 1, 30

4, 3, 6

0, 4, 5

dist =

0	1	2	3	4
0	17	13	11	5

n = 5 ✗ ✗ ✗ ✓

{ dist[u] + wt < dist[v]

Ques:

Q : Network Delay Time

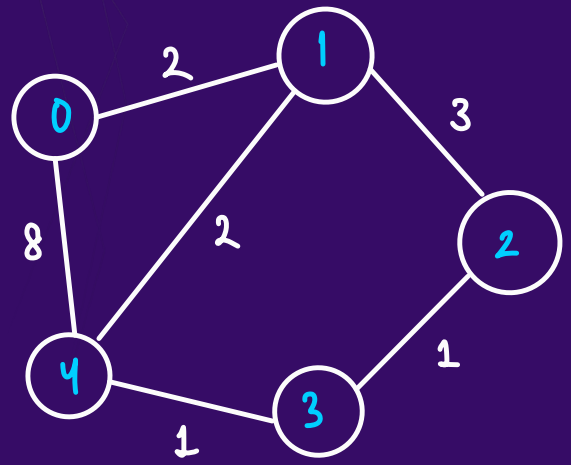
[Leetcode 743]

Floyd-Warshall Algorithm

A to B through 4

PW SKILLS

All Pairs Shortest Path



	0	1	2	3	4
0	0	2	5	5	4
1	2	0	3	3	2
2	5	3	0	1	2
3	5	3	1	0	1
4	4	2	2	1	0

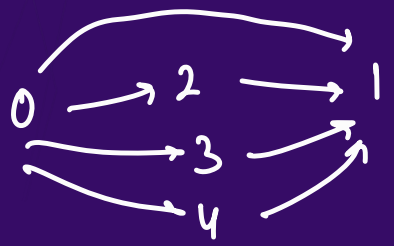
adj

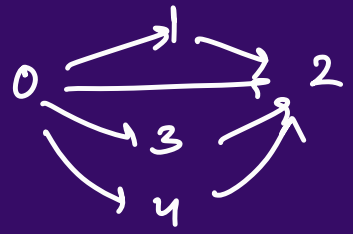
Go from A to B via/through C.

A,B = min(A,B , A,C + C,B)

Floyd-Warshall Algorithm

All Pairs Shortest Path





$0 \to 3 \to 2 \quad \to \quad 0 \to 3 + 3 \to 2$

Floyd-Warshall Algorithm

PW SKILLS

All Pairs Shortest Path

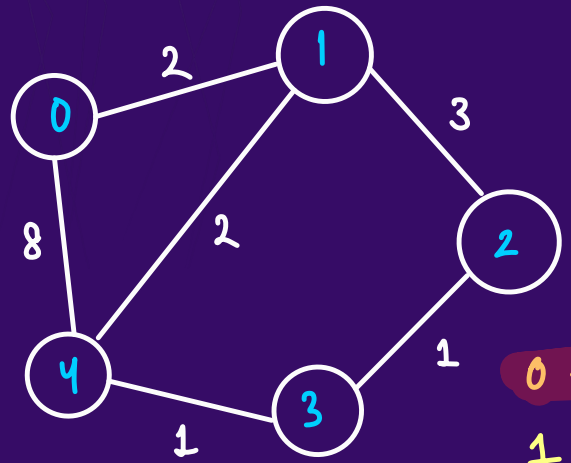
n nodes from 0, 1

```
for(int k=0; k<n; k++){ //through k
    for(int i=0; i<n; i++){
        for(int j=0; j<n; i++){
            dis[i][j] = min(dis[i][j], dist[i][k] + dis[k][j])
        }
    }
}
```

T.C. = $O(n^3)$ S.C. = $O(n^2)$

Ques:

Q : Find the City with smallest number of neighbours at a Threshold distance



t = 2

	0	1	2	3	4
0	0	2	5	5	4
1	2	0	3	3	2
2	5	3	0	1	2
3	5	3	1	0	1
4	4	2	2	1	0

0 → 1

1 → 0, 4

2 → 3, 4

3 → 2, 4

4 → 1, 2, 3

[Leetcode 1334]